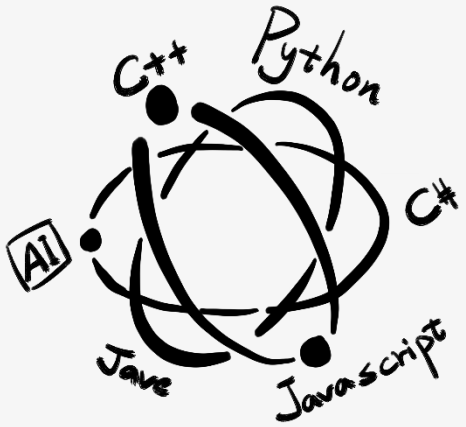




区间问题

酷哥创客 COOLMAKER







1 温故知新

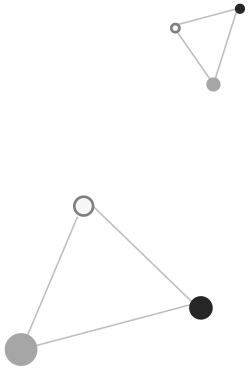
2 知识宝典

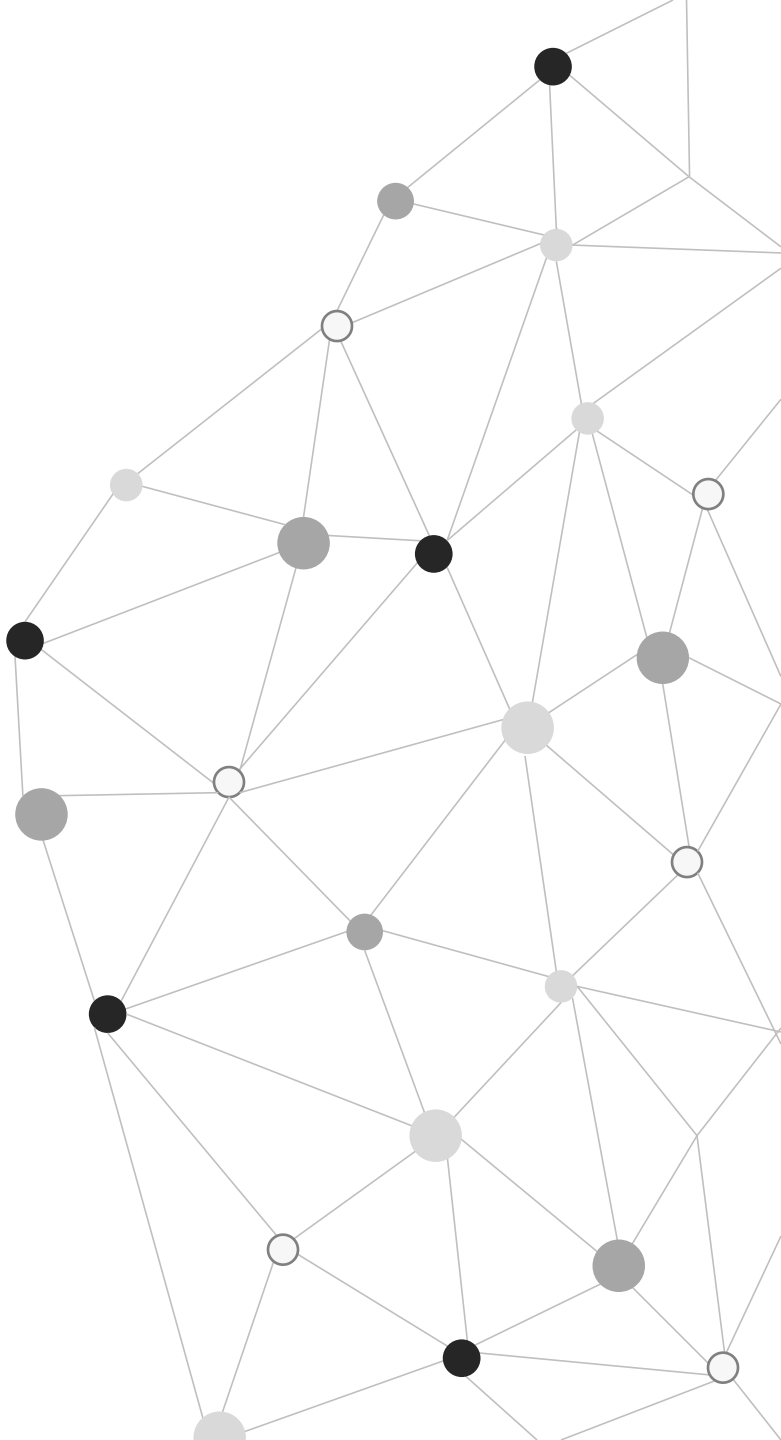
3 运用自如

4 回顾总结



知 识 宝 典







COOLMAKER
酷哥创客 人工智能教育

- 【5】双端栈
- 【5】双端队列
- 【5】单调队列
- 【6】优先队列
- 【6】ST表（Sparse Table）
- 【6】树状数组
- 【6】线段树





ST表



ST表的引入

COOLMAKER
酷哥创客 人工智能教育



ST表（Sparse Table，**稀疏表**）是一种简单的数据结构。

主要用来解决**静态RMQ**（Range Maximum/Minimum Query，**区间最大/最小值查询**）等**可重复贡献问题**。它主要应用**倍增和动态规划**的思想，可以实现O(n log n)**预处理**、O(1)**查询**，**不支持修改**。常数远小于树状数组、线段树等数据结构。



倍增，顾名思义，就是成倍的增加，能够使得**线性的处理**转化为**对数级的处理**，大大地优化时间复杂度



ST表的引入

COOLMAKER
酷哥创客 人工智能教育

可重复贡献问题是指对于运算 opt，**满足 x opt x=x**，则对应的区间询问就是一个可重复贡献问题。例如，最大值有 max(x,x)=x，gcd 有 gcd(x,x)=x，所以 RMQ 和区间 GCD 就是一个可重复贡献问题。像区间和就不具有这个性质，如果求区间和的时候采用的预处理区间重叠了，则会导致重叠部分被计算两次，这是我们所不愿意看到的。另外，opt 还必须**满足结合律才能使用 ST 表求解。**

形象地说，我们要查询一块区域A内的元素性质，可以将该区域用两块区域B,C来代替；这两个区间可以包含相同元素，但不可以包含不属于A的元素，即 B ∪ C=A且B ∩ C∈A。

int a[]={**1, 9, 4, 3, 1, 5**, 4, 3, 6, 12};

例如，求一个数组的最大值，比如说长度为10的数组，我们可以先求前6个数的最大值，再求后7个数的最大值，最后求这两个最大值的最大值，虽然这中间有重复的元素，但是对最终的最大值结果不会有影响，这就**叫做可重复贡献问题。**

常见的可重复贡献问题有：**区间最值、区间按位和、区间按位或、区间GCD**等。而像**区间和**这样的问题就不是可重复贡献问题。

ST表的引入

COOLMAKER
酷哥创客 人工智能教育

3284: 区间最大值

时间限制: 2 Sec 内存限制: 256 MB
提交: 0 解决: 1
[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★☆☆

题目描述

输入一串数字，给你 M 个询问，每次询问就给你两个数字 X, Y，要求你说出 X 到 Y 这段区间内的最大数。

输入

第一行两个整数 N, M 表示数字的个数和要询问的次数;
接下来一行为 N 个数;
接下来 M 行，每行都有两个整数 X, Y。

对于全部数据，$1 \leq N \leq 10^5, 1 \leq M \leq 10^6, 1 \leq X \leq Y \leq N$。数字不超过 C/C++ 的 int 范围。

输出

输出共 M 行，每行输出一个数。

样例输入

```
10 2
3 2 4 5 6 8 1 2 9 7
1 4
3 8
```

样例输出

```
5
8
```

提示

静态区间最大值

ST表的引入

COOLMAKER
酷哥创客 人工智能教育

题目大意：给定 n 个数，有 m 个询问，对于每个询问，你需要回答区间 [l,r] 中的最大值。

考虑暴力做法。每次都对区间 [l,r] 扫描一遍，求出最大值。

对于m次询问，每个区间长度最大为n，时间复杂度为**O(n*m)**，显然，这个算法会超时。

```
while(m--)
{
    cin>>l>>r;
    int mx=a[l];
    for(int i=l+1;i<=r;i++)
    {
        mx=max(mx,a[i]);
    }
    cout<<mx<<"\n";
}
```



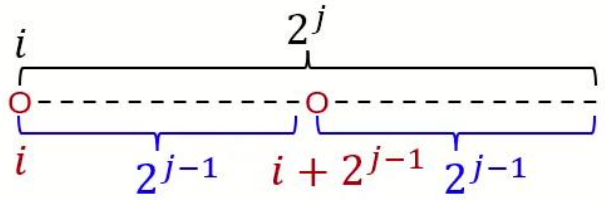
ST表的原理

COOLMAKER 酷哥创客 人工智能教育

预处理阶段(构建)：

倍增法递推：用两个等长小区间拼凑一个大区间

$f[i][j]$ 以第 i个数为起点，长度为 2^j 的区间中的最大值。



$$f[i][j]=\max(f[i][j-1],f[i+2^{j-1}][j-1])$$

```
for(int i=1;i<=n;i++) scanf("%d",&f[i][0]);
for(int j=1;j<=20;j++) //枚举区间长度
  for(int i=1;i+(1<<j)-1<=n;i++) //枚举起点
    f[i][j]=max(f[i][j-1],f[i+(1<<(j-1))][j-1]);
```

区间终点：$i+2^j-1\le n$

$n=6$

区间长度倍增：1,2,4,8,...

$f[i,0]$: [1 1][2 2][3 3][4 4][5 5][6 6]

$f[i,1]$: [1 2][2 3][3 4][4 5][5 6]

$f[i,2]$: [1 4][2 5][3 6]

$j=3$: $i+2^j-1=1+8-1>6$

ST表的原理

COOLMAKER
酷哥创客 人工智能教育

询问阶段：

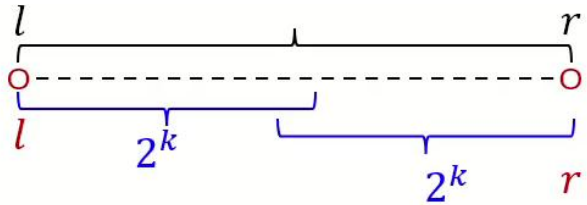
对查询区间 $[l,r]$ 做分割、拼凑

区间长度的指数 $k=\log2(r-l+1)$

$k=0$: $\{1\}$
$k=1$: $\{2,3\}$
$k=2$: $\{4,5,6,7\}$
$k=3$: $\{8,9,10,\ldots,15\}$

观察发现：$2^k \le r-l+1 < 2\cdot 2^k$，

即区间 $[l,r]$ 必可以用两个长度为 2^k 的区间**重叠拼凑**。



即 $\max(f[l][k], f[r-2^k+1][k])$

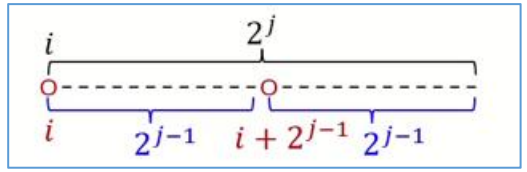
```
for(int i=1,l,r;i<=m;i++){
  scanf("%d%d",&l,&r);
  int k=log2(r-l+1); //区间长度的指数
  printf("%d\n",max(f[l][k],f[r-(1<<k)+1][k]));
}
```

$[1,4] \rightarrow [1,4]+[1,4]$

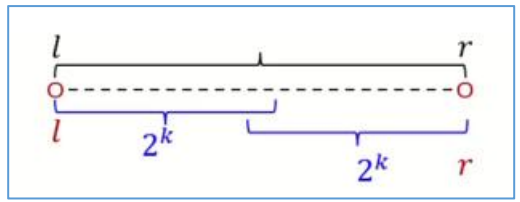
$[1,5] \rightarrow [1,4]+[2,5]$

$[1,6] \rightarrow [1,4]+[3,6]$

$[1,7] \rightarrow [1,4]+[4,7]$




```
void build()//构建st表
{
    for(int i=2; i<=n+1; i++)lg2[i]=lg2[i/2]+1;//预处理log2
    for(int i=1; i<=n; i++)//区间长度为1，先填进st表中
    {
        dp[i][0]=a[i];
    }
    for(int j=1; j<=20; j++)//倍增区间长度
    {
        for(int i=0; i+(1<<j)-1<=n; i++)
        {
            dp[i][j]=max(dp[i][j-1],dp[i+(1<<(j-1))][j-1]);
        }
    }
}
```




```
int query(int l,int r)//查询
{
    int k=lg2[r-l+1];
    return max(dp[l][k],dp[r-(1<<k)+1][k]);
}
```

```
#include <bits/stdc++.h>
using namespace std;
const int maxn =1e5+5;
int n,m,x,y,a[maxn];
struct ST
{
    int dp[maxn][20],lg2[maxn];
    void build()//构建st表
    {...}
    int query(int l,int r)//查询
    {...}
} st;
void solve()
{
    cin>>n>>m;
    for(int i=1; i<=n; i++)
    {
        cin>>a[i];
    }
    st.build();
    while(m--)
    {
        cin>>x>>y;
        cout<<st.query(x,y)<<"\n";
    }
}
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    solve();
    return 0;
}
```

2626: 策略游戏

COOLMAKER 人工智能教育

题目描述

小 L 和小 Q 在玩一个策略游戏。

有一个长度为 n 的数组 A 和一个长度为 m 的数组 B，在此基础上定义一个大小为 $n \times m$ 的矩阵 C，满足 $C_{ij} = A_i \times B_j$。所有下标均从 1 开始。

游戏一共会进行 q 轮，在每一轮游戏中，会事先给出 4 个参数 l_1, r_1, l_2, r_2，满足 $1 \leq l_1 \leq r_1 \leq n$、$1 \leq l_2 \leq r_2 \leq m$。

游戏中，小 L 先选择一个 $l_1 \sim r_1$ 之间的下标 x，然后小 Q 选择一个 $l_2 \sim r_2$ 之间的下标 y。定义这一轮游戏中二人的得分是 C_{xy}。

小 L 的目标是使得这个得分尽可能大，小 Q 的目标是使得这个得分尽可能小。同时两人都是足够聪明的玩家，每次都会采用最优的策略。

请问：按照二人的最优策略，每轮游戏的得分分别是多少？

输入

第一行输入三个正整数 n, m, q，分别表示数组 A，数组 B 的长度和游戏轮数。

第二行：n 个整数，表示 A_i，分别表示数组 A 的元素。

第三行：m 个整数，表示 B_i，分别表示数组 B 的元素。

接下来 q 行，每行四个正整数，表示这一次游戏的 l_1, r_1, l_2, r_2。

输出

输出共 q 行，每行一个整数，分别表示每一轮游戏中，小 L 和小 Q 在最优策略下的得分。

2626: 策略游戏

COOLMAKER
酷哥创客 人工智能教育

样例输入1 复制

```
3 2 2
0 1 -2
-3 4
1 3 1 2
2 3 2 2
```

样例输出1 复制

```
0
4
```

【样例解释 #1】

这组数据中，矩阵 C 如下：

$$\begin{bmatrix} 0 & 0 \\ -3 & 4 \\ 6 & -8 \end{bmatrix}$$

在第一轮游戏中，无论小 L 选取的是 $x = 2$ 还是 $x = 3$，小 Q 都有办法选择某个 y 使得最终的得分为负数。因此小 L 选择 $x = 1$ 是最优的，因为这样得分一定为 0。

而在第二轮游戏中，由于小 L 可以选 $x = 2$，小 Q 只能选 $y = 2$，如此得分为 4。

【数据范围】

对于所有数据，$1 \le n, m, q \le 10^5$，$-10^9 \le A_i, B_i \le 10^9$。对于每轮游戏而言，$1 \le l_1 \le r_1 \le n$，$1 \le l_2 \le r_2 \le m$。

给定两个区间，先手在 A 序列给定区间内选择一个数，后手在 B 序列给定区间内选另一个数。两人所选数字乘积为得分。先手希望得分尽量大，后手希望得分尽量小。

【数据范围】

对于所有数据，$1 \le n, m, q \le 10^5$，$-10^9 \le A_i, B_i \le 10^9$。对于每轮游戏而言，$1 \le l_1 \le r_1 \le n$，$1 \le l_2 \le r_2 \le m$。

测试点编号	$n, m, q \le$	特殊条件
1	200	1, 2
2	200	1
3	200	2
$4 \sim 5$	200	无
6	1000	1, 2
$7 \sim 8$	1000	1
$9 \sim 10$	1000	2
$11 \sim 12$	1000	无
13	10^5	1, 2
$14 \sim 15$	10^5	1
$16 \sim 17$	10^5	2
$18 \sim 20$	10^5	无

其中，特殊性质 1 为：保证 $A_i, B_i > 0$。

特殊性质 2 为：保证对于每轮游戏而言，要么 $l_1 = r_1$，要么 $l_2 = r_2$。

特殊性质一：权值为正

因为不存在相乘正负性改变的问题，先手必然选择区间内最大值，后手必然选择区间内最小值。

给定两个区间，先手在 A 序列给定区间内选择一个数，后手在 B 序列给定区间内选另一个数。两人所选数字乘积为得分。先手希望得分尽量大，后手希望得分尽量小。

【数据范围】

对于所有数据，$1 \le n, m, q \le 10^5$，$-10^9 \le A_i, B_i \le 10^9$。对于每轮游戏而言，$1 \le l_1 \le r_1 \le n$，$1 \le l_2 \le r_2 \le m$。

测试点编号	$n, m, q \le$	特殊条件
1	200	1, 2
2	200	1
3	200	2
$4 \sim 5$	200	无
6	1000	1, 2
$7 \sim 8$	1000	1
$9 \sim 10$	1000	2
$11 \sim 12$	1000	无
13	10^5	1, 2
$14 \sim 15$	10^5	1
$16 \sim 17$	10^5	2
$18 \sim 20$	10^5	无

其中，特殊性质 1 为：保证 $A_i, B_i > 0$。

特殊性质 2 为：保证对于每轮游戏而言，要么 $l_1 = r_1$，要么 $l_2 = r_2$。

特殊性质二：存在一方的操作区间长度为 1

若先手操作区间长度为 1，则有如下几种情况：

先手只能选的数是 0，此时答案是 0。

先手只能选的数是正数，此时后手策略为选择其区间内的最小数。

先手只能选的数是负数，此时后手策略为选择其区间内的最大数。

后手同理：

后手只能选的数是 0，此时答案是 0。

后手只能选的数是正数，此时先手策略为选择其区间内的最大数。

后手只能选的数是负数，此时后手策略为选择其区间内的最小数。

给定两个区间，先手在 A 序列给定区间内选择一个数，后手在 B 序列给定区间内选另一个数。两人所选数字乘积为得分。先手希望得分尽量大，后手希望得分尽量小。

【数据范围】

对于所有数据，$1 \le n, m, q \le 10^5$，$-10^9 \le A_i, B_i \le 10^9$。对于每轮游戏而言，$1 \le l_1 \le r_1 \le n$，$1 \le l_2 \le r_2 \le m$。

测试点编号	$n, m, q \le$	特殊条件
1	200	1, 2
2	200	1
3	200	2
$4 \sim 5$	200	无
6	1000	1, 2
$7 \sim 8$	1000	1
$9 \sim 10$	1000	2
$11 \sim 12$	1000	无
13	10^5	1, 2
$14 \sim 15$	10^5	1
$16 \sim 17$	10^5	2
$18 \sim 20$	10^5	无

这其实就是启示我们要思考两人博弈的策略

其中，特殊性质 1 为：保证 $A_i, B_i > 0$。

特殊性质 2 为：保证对于每轮游戏而言，要么 $l_1 = r_1$，要么 $l_2 = r_2$。

A 在 $a[l_1 \cdots r_1]$ 中选择一个 x，然后 B 在 $b[l_2 \cdots r_2]$ 中选择一个 y，分数是 $x \times y$，A 想让分数尽可能大，B 想让分数尽可能小。求分数。

肯定先思考 B 再思考 A，因为 A 会思考 B 的思考。

B 的行为就是对于 x，找到一个 $b[l_2 \cdots r_2]$ 中的 y，使得 $x \times y$ 最小。

具体地：

- $x \geq 0$ 时，B 会选择最小的 y；
- $x < 0$ 时，B 会选择最大的 y。

所以第二个人不用考虑他的数的正负，而第一个人就需要考虑这几种情况选最大值：
自己最大的正数与对面最小的数的积；
自己最小的正数与对面最小的数的积；
自己最大的负数与对面最大的数的积；
自己最小的负数与对面最大的数的积；

因此，我们维护 A 的四种情况的值和 B 的两种情况的值，通过上述四个式子计算答案。





COOLMAKER
酷哥创客 人工智能教育

暴力部分分：

```cpp
#include <bits/stdc++.h>
using namespace std;
int n,m,q,a[10010],b[10010];
long long c[10010][10010],ans,num;
int l1,l2,r1,r2;
int main()
{
    ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    //freopen("game.in","r",stdin);
    //freopen("game.out","w",stdout);
    cin>>n>>m>>q;
    for(int i=1;i<=n;i++) cin>>a[i];     //nx1列向量
    for(int i=1;i<=m;i++) cin>>b[i];     //1xn行向量
    for(int i=1;i<=n;i++)                //nxn矩阵
        for(int j=1;j<=m;j++)
            c[i][j]=(long long)a[i]*(long long)b[j];
    for(int k=1;k<=q;k++)
    {
        cin>>l1>>r1>>l2>>r2;
        ans=-0x3f3f3f3f3f3f3f3f;
        num=0x3f3f3f3f3f3f3f3f;
        for(int i=l1;i<=r1;i++)
        {
            num=0x3f3f3f3f3f3f3f3f;
            for(int j=l2;j<=r2;j++)
            {
                num=min(c[i][j],num);
            }
            ans=max(ans,num);
        }
        cout<<ans<<"\n";
    }
    return 0;
}
```

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 5;
int lg2[N];
int afmin[N][20], afmax[N][20], azmin[N][20], azmax[N][20];
int bmin[N][20], bmax[N][20];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    int n, m, q;
    cin >> n >> m >> q;
    for(int i = 1, x; i <= n; i++)
    {
        cin >> x;
        afmin[i][0] = (x < 0 ? x : 0);
        afmax[i][0] = (x < 0 ? x : INT_MIN);
        azmin[i][0] = (x >= 0 ? x : INT_MAX);
        azmax[i][0] = (x >= 0 ? x : -1);
    }
    for(int i = 1, x; i <= m; i++)
    {
        cin >> x;
        bmin[i][0] = bmax[i][0] = x;
    }
    for(int i = 2; i < N; i++)
        lg2[i] = lg2[i >> 1] + 1;
    for(int j = 1; j <= 19; j++)
    {
        for(int i = 1; i + (1 << j) - 1 < N; i++)
        {
            int p = i + (1 << (j - 1)); // [i,p-1]  [p,i+(1<<j)-1];
            afmin[i][j] = min(afmin[i][j - 1], afmin[p][j - 1]);
            afmax[i][j] = max(afmax[i][j - 1], afmax[p][j - 1]);
            azmin[i][j] = min(azmin[i][j - 1], azmin[p][j - 1]);
            azmax[i][j] = max(azmax[i][j - 1], azmax[p][j - 1]);
            bmin[i][j] = min(bmin[i][j - 1], bmin[p][j - 1]);
            bmax[i][j] = max(bmax[i][j - 1], bmax[p][j - 1]);
        }
    }
    while(q--)
    {...}
    return 0;
}
```

```cpp
    while(q--)
    {
        int l1, l2, r1, r2;
        cin >> l1 >> r1 >> l2 >> r2;
        int k = lg2[r1 - l1 + 1];
        long long a1 = min(afmin[l1][k], afmin[r1 - (1 << k) + 1][k]);
        long long a2 = max(afmax[l1][k], afmax[r1 - (1 << k) + 1][k]);
        long long a3 = min(azmin[l1][k], azmin[r1 - (1 << k) + 1][k]);
        long long a4 = max(azmax[l1][k], azmax[r1 - (1 << k) + 1][k]);
        k = lg2[r2 - l2 + 1];
        long long b1 = min(bmin[l2][k], bmin[r2 - (1 << k) + 1][k]);
        long long b2 = max(bmax[l2][k], bmax[r2 - (1 << k) + 1][k]);
        long long ans = LONG_LONG_MIN;
        if(a1 != 0)
        {
            ans = max(ans, a1 * b2);
            ans = max(ans, a2 * b2);
        }
        if(a3 != INT_MAX)
        {
            ans = max(ans, a3 * b1);
            ans = max(ans, a4 * b1);
        }
        cout << ans << "\n";
    }
    return 0;
}
```

2668 问题 B: 静态区间

时间限制: 1 Sec 内存限制: 128 MB

提交: 0 解决: 2

[提交] [状态] [讨论版] [命题人: hjw] [Edit] [TestData]

难度

★★★☆☆

题目描述

给定一行n个正整数a1……an。

m次询问，每次询问给定一个区间[l,r]，输出al……ar的最大公因数。

输入

第一行两个整数n，m。

第二行n个整数表示a1……an。

以下m行，每行2个整数表示询问区间的左右端点。

对于100%的数据$1\le n\le 10^5$，$1\le m\le 10^5$，$0<ai\le 10^9$

保证输入数据合法。

输出

输出共m行，每行输出一个数。

样例输入

```
5 3
4 12 3 6 7
1 3
2 3
5 5
```

样例输出

```
1
3
7
```

```
struct ST
{
    int n;
    int dp[maxn][20];
    int lg2[maxn];
    void build()
    {
        for(int i=2; i<=n+1; i++)lg2[i]=lg2[i/2]+1;
        for(int i=1; i<=n; i++)
        {
            dp[i][0]=a[i];
        }
        for(int j=1; j<=20; j++)
        {
            for(int i=0; i+(1<<j)-1<=n; i++)
            {
                dp[i][j]=__gcd(dp[i][j-1],dp[i+(1<<(j-1))][j-1]);
            }
        }
    }
    int query(int l,int r)
    {
        int k=lg2[r-l+1];
        return __gcd(dp[l][k],dp[r-(1<<k)+1][k]);
    }
} st;
```

3310 问题 F: 01 序列

时间限制: 3 Sec 内存限制: 512 MB
提交: 0 解决: 3
[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★★☆

对于查询1，求区间的最长不下降子序列
由于序列只存在01两种数值，序列 0101 的不下降子序列有 1 、 0 、 01 、 011 、 001这几种

每个不下降子序列必定是由几个0后面跟着几个1构成的
设置数组pre0来统计以i为结尾的前缀0的数量
设置数组suf1来统计以i为开始的后缀1的数量

对于区间[L,R]内的一个位置i，以它为01的分界线的不下降子序列长度为
(pre[i]-pre[L-1])+(suf[i]-suf[R+1])
那么整个区间的最长不下降子序列长度就是对于每一个位置i，根据上述公式取最大值
max {(pre[i]-pre[L-1])+(suf[i]-suf[R+1]) | i属于[L,R]}
暴力寻找一个区间中的答案的时间复杂度是O(n)的，m次查询，这样总体时间复杂度就是O(n*m)，会超时

由于ST表初始化时的每一个位置的值只能和其本身有关，尝试把上述公式化成只和 i 有关
max {pre[i]+suf[i] | i属于[L,R]} - （pre[L-1]+suf[R+1]）
ST中找区间[L，R]中的pre[i]+suf[i]最大值，减去（pre[L-1]+suf[R+1]）就是查询1的答案了
寻找一个区间中的答案的时间复杂度是O(1)的，构建ST表复杂度是O(nlogn)的
此时的总体时间复杂度就是O(nlogn+m)

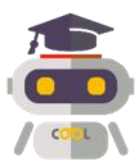
```
2
2
1
```

间没有新的01连续序列，输出1
否则代表有此时输出2
此时单次查询2的时间复杂度是O(1)

提示
对于查询1，考虑前缀和，后缀和，数学公式推导，推导出，什么部分可以放到st表中
对于查询2，显然只有1，2这两种答案，想一想这两种答案的情况是什么，考虑和前缀和结合

```cpp
void build()//构建st表
{
    for(int i=2; i<=n+1; i++)lg2[i]=lg2[i/2]+1;//预处理log2
    for(int i=1; i<=n; i++)//区间长度为1，先填进st表中
    {
        dp[i][0]=pre[i]+suf[i];
    }
    for(int j=1; j<=20; j++)//倍增区间长度
    {
        for(int i=0; i+(1<<j)-1<=n; i++)
        {
            dp[i][j]=max(dp[i][j-1],dp[i+(1<<(j-1))][j-1]);
        }
    }
}
```

```cpp
int query(int l,int r)//查询
{
    int k=lg2[r-l+1];
    return max(dp[l][k],dp[r-(1<<k)+1][k]);
}
```



```cpp
#include <bits/stdc++.h>
using namespace std;
const int maxn =1e5+5;
int n,m,op,x,y,a[maxn];
int pre[maxn],suf[maxn],sum01[maxn];
struct ST
{
    int dp[maxn][20],lg2[maxn];
    void build()//构建st表
    {...}
    int query(int l,int r)//查询
    {...}
} st;
void solve()
{
    cin>>n>>m;
    for(int i=1; i<=n; i++)
    {
        cin>>a[i];
        if(i>1)sum01[i]=sum01[i-1]+(a[i]==1&&a[i-1]==0);
        pre[i]=pre[i-1]+(a[i]==0);
    }
    for(int i=n;i>=1;i--)
    {
        suf[i]=suf[i+1]+(a[i]==1);
    }
    st.build();
    while(m--)
    {
        cin>>op>>x>>y;
        if(op==1)
        {
//max{(pre0[i]-pre0[L-1])+(suf1[i]-suf1[R+1])}==(-suf1[R+1]-pre0[L-1])+max(pre0[i]+suf1[i])
//所以就是求[L,R]中最大的pre0[i]+suf1[i]
            cout<<st.query(x,y)-pre[x-1]-suf[y+1]<<"\n";
        }else
        {
            cout<<1+(sum01[y]-sum01[x]>0)<<"\n";
        }
    }
}
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    solve();
    return 0;
}
```

来统计以i为结尾包含的01连续序列数量

来统计以i为结尾前缀0个数

来统计以i为起始的后缀1个数

2670 问题 H: 矩阵最值

时间限制: 2 Sec 内存限制: 512 MB

提交: 0 解决: 2

[提交] [状态] [讨论版] [命题人: hjw] [Edit] [TestData]

难度

★ ★ ★ ★ ★

题目描述

我们有一个n行m列的矩阵，现在小Q有K个问题，每次询问一个以(x1,y1)为左上角(x2,y2)，为右下角的子矩阵的最大值。

输入

第一行三个整数n,m,K。

接下来n行，每行有m个整数，设a_{i,j}为矩阵i行j列的数字。

接下来K行，每行4个整数x1,y1,x2,y2，保证1≤x1≤x2≤n，1≤y1≤y2≤m。

对于100%的数据，1≤n,m≤250，1≤k≤10^6

输出

共K行，每行对应一个答案。

样例输入

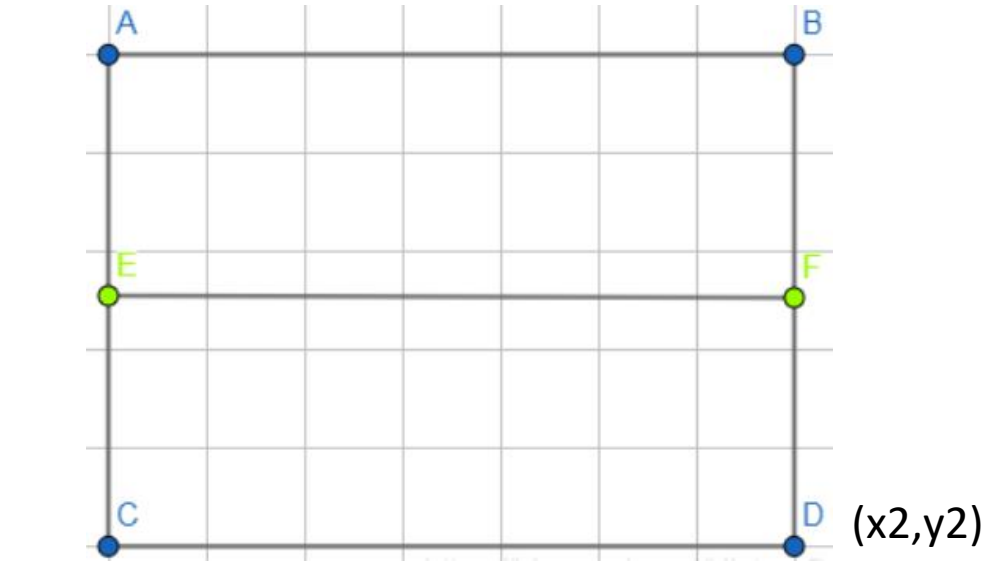
```
3 4 5
789 15225 27847 6452
3976 18268 23626 1943
13336 26216 17321 4960
2 2 3 4
2 3 3 4
2 1 3 4
1 3 2 4
1 2 3 2
```

样例输出

```
26216
23626
26216
27847
26216
```

```cpp
#include <bits/stdc++.h>
using namespace std;
const int maxn =250+5;
int n,m,q,a,b,c,d;
int val[maxn][maxn];
struct ST_2D
{
    int dp[maxn][maxn][9][9];
    int lg2[maxn];
    void build()
    {...}
    int query(int x1,int y1,int x2,int y2)
    {...}
} st;
void solve()
{
    cin>>n>>m>>q;
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=m; j++)
        {
            cin>>val[i][j];
        }
    }
    st.build();
    while(q--)
    {
        cin>>a>>b>>c>>d;
        cout<<st.query(a,b,c,d)<<"\n";
    }
}
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    solve();
    return 0;
}
```

```
void build()
{
    for(int i=2; i<=250; i++)lg2[i]=lg2[i/2]+1;//预处理log2
    for(int K=0; K<=8; K++)
    {
        for(int L=0; L<=8; L++)
        {
            for(int i=1; i+(1<<K)-1<=n; i++)
            {
                for(int j=1; j+(1<<L)-1<=m; j++)
                {
                    if(!K&&!L)dp[i][j][0][0]=val[i][j];
                    else if(L)  //L>0
                    {
                        dp[i][j][K][L]=max(dp[i][j][K][L-1],dp[i][j+(1<<(L-1))][K][L-1]);
                    }
                    else        //K>0
                    {
                        dp[i][j][K][L]=max(dp[i][j][K-1][L],dp[i+(1<<(K-1))][j][K-1][L]);
                    }
                }
            }
        }
    }
}
```



那么我们将点E看作$(i, j+2^{l-1})$，其实原来的大矩阵就可以由分成的这两个小矩阵更新得到，转移如下：

$$st[i][j][k][l] = \max\{st[i][j][k][l-1], st[i][j+2^{l-1}][k][l-1]\}$$

其中，max里面第一个为上半部分矩阵，后面一个为下半部分矩阵。

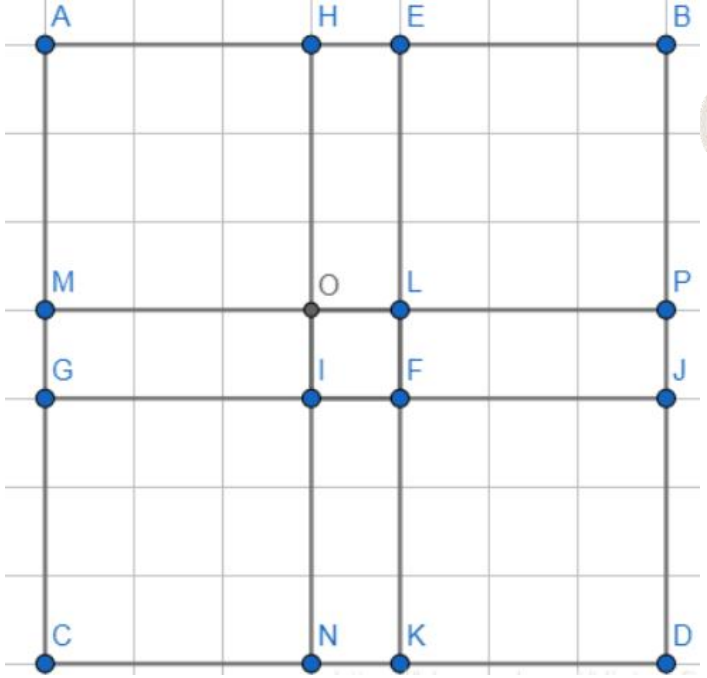
如果$l = 0$的话，就将其竖起剖成两部分即可，是同理的。转移如下：

$$st[i][j][k][l] = \max\{st[i][j][k-1][l], st[i+2^{k-1}][j][k-1][l]\}$$

所以最后按照k, l从小到大更新即可。

```
int query(int x1,int y1,int x2,int y2)
{
    int p=lg2[x2-x1+1],q=lg2[y2-y1+1];
    return max({dp[x1][y1][p][q]
                ,dp[x2-(1<<p)+1][y1][p][q]
                ,dp[x1][y2-(1<<q)+1][p][q]
                ,dp[x2-(1<<p)+1][y2-(1<<q)+1][p][q]});
}
```

(x1,y1)



(x2,y2)

COOLMAKER
酷哥创客 人工智能教育

其中四个部分为图中$W_1(A,E,F,G), W_2(H,B,J,I), W_3(M,L,K,C), W_4(O,P,D,N)$，查询区间是可以重合的。

其实就对应了如下四个预处理的状态，我们令$p = log_2(x_2 - x_1 + 1), q = log_2(y_2 - y_1 + 1)$:

$$\max \begin{cases} st[x_1][y_1][p][q] & \rightarrow & W_1 \\ st[x_2 - 2^p + 1][y_1][p][q] & \rightarrow & W_2 \\ st[x_1][y_2 - 2^q + 1][p][q] & \rightarrow & W_3 \\ st[x_2 - 2^p + 1][y_2 - 2^q + 1] & \rightarrow & W_4 \end{cases} \rightarrow ans$$

答案就为上面四个矩阵的max，所以预处理对数，每次$O(1)$回答即可。

ST表的局限性

凡是符合**结合律**且**可重复贡献**的信息查询都可以使用ST表。显然最大值、最小值、最大公因数、最小公倍数、按位或按位与 都符合这个条件。

ST最大的局限有两点：
（1）不支持修改
（2）如果区间不可重叠（如区间乘），则不可使用ST表

如果**涉及修改操作**，就要使用**线段树**。





线段树





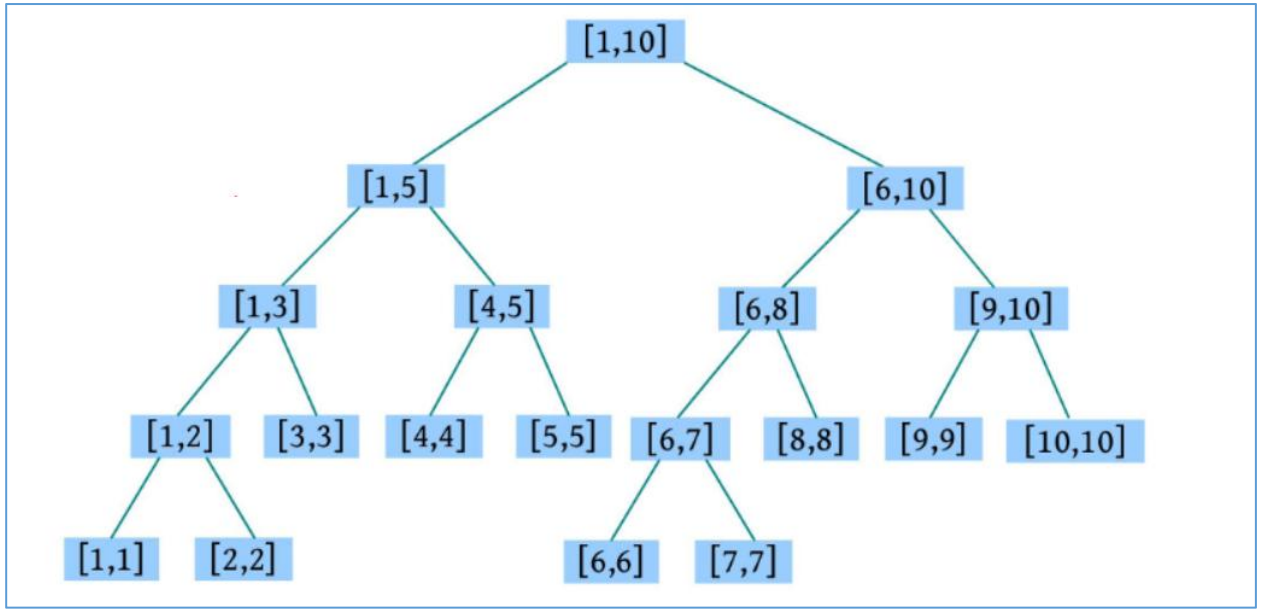
线段树的引入

COOLMAKER
酷哥创客 人工智能教育

线段树是一种基于「分治思想」的二叉树。

每次把区间 [l ,r] 分成左右两个子区间，左节点区间 [l, (l + r)/2]，右节点区间 [(l + r)/2+1, r] ，从而把整个线段分为一个树形结构。

递归求解，通过合并左右两子节点区间信息至父节点来求得该区间的信息。这种数据结构可以方便的进行大部分的区间操作。

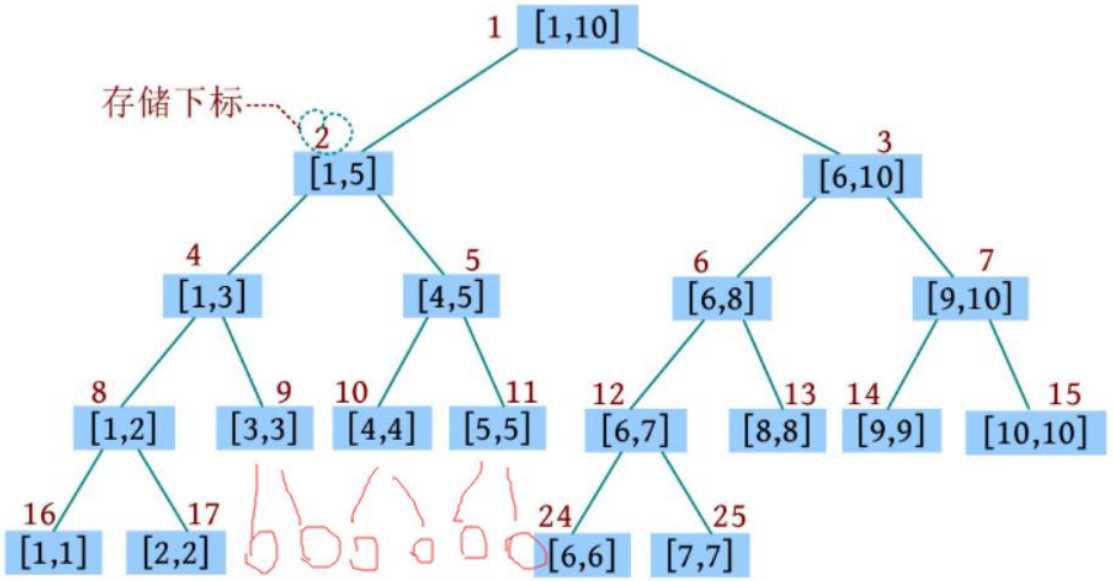


线段树的引入

COOLMAKER 酷哥创客 人工智能教育

线段树除了最后一层，其他层构成一颗满二叉树，因此可以采用顺序存储方式，用一个数组tree[]存储节点。如果一个节点的存储下标为k，则其左子节点的下标为2k，其右子节点的下标为2k+1。

注意：这里使用2k，2k+1作为访问左右孩子，是有空间消耗的，一般而言只有一棵树是完全二叉树才可以使用2k，2k+1表示左右孩子，从下图中可以看到，区间[6,6]如果想要被2k进行访问，实际上区间[3,3] [4,4] [5,5]也需要创建相应的空叶子节点



线段树的引入

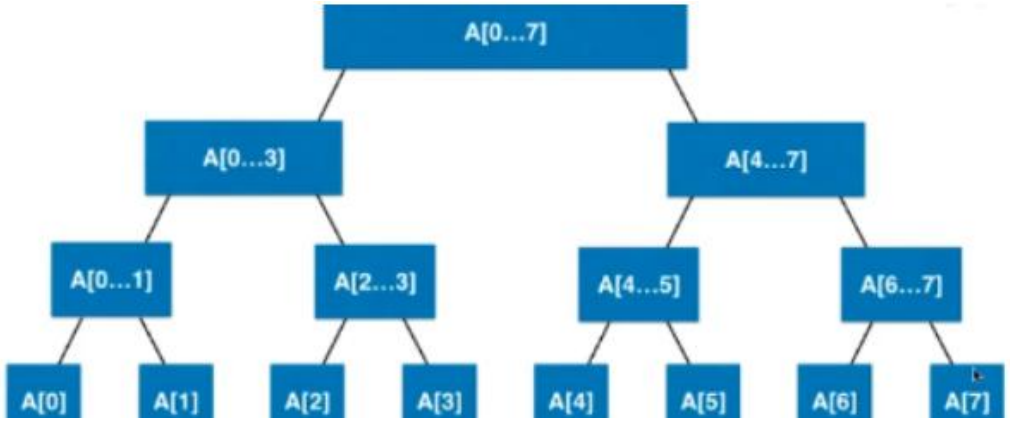
COOLMAKER 酷哥创客 人工智能教育

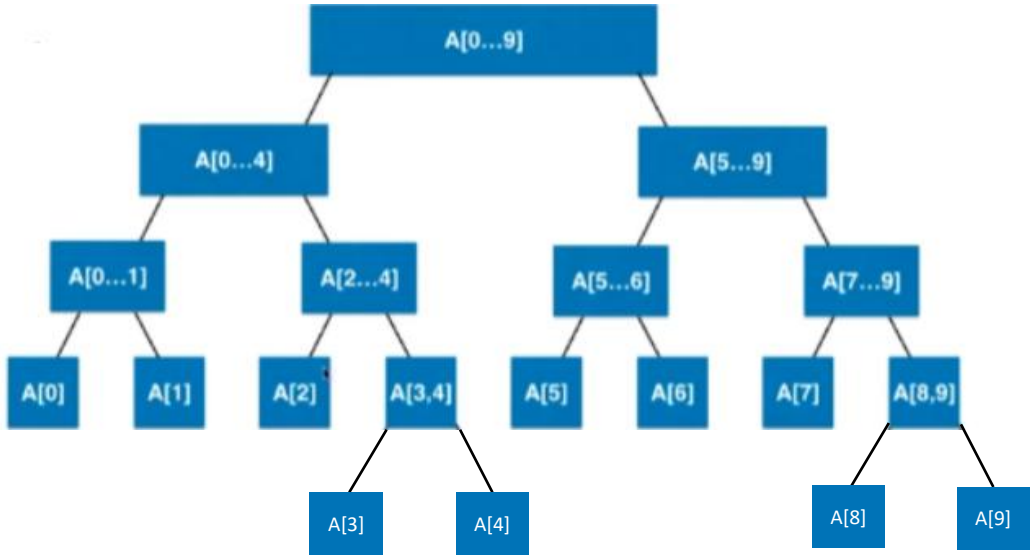
那么既然有空间消耗，一般我们需要开多少的空间才能支持线段树呢？

假设区间长度 $n = 2^k$，那么总的结点个数为 $2^{k+1} - 1$（满二叉树），所以此时我们只需要开**2n**的空间即可

当区间长度 $n != 2^k$时，就需要额外多出一层来存储前面的空结点，此时我们就需要开**4n**的空间。

所以一般定义tree数组的时候需要开始**4倍**的空间。





线段树的单点修改、区间查询

首先我们来看一道关于可以用线段树来解决的**单点修改**、**区间查询**的问题



3494: 【模板】线段树单点修改、区间查询

时间限制: 1 Sec 内存限制: 128 MB

提交: 0 解决: 1

[提交] [状态] [讨论版] [命题人: 13305755511] [Edit] [TestData]

难度

★★★☆☆

题目描述

这是一道模板题。

给定数列 $a_1, a_2, \ldots, a_n$，你需要依次进行 q 个操作，操作有两类：

- `1 i x`：给定 i, x，将 a_i 加上 x；
- `2 l r`：给定 l, r，求 $\sum_{i=l}^{r} a_i$ 的值（换言之，求 $a_l + a_{l+1} + \cdots + a_r$ 的值）。

输入

第一行包含 2 个正整数 n, q，表示数列长度和询问个数。保证 $1 \le n, q \le 10^6$。
第二行 n 个整数 $a_1, a_2, \ldots, a_n$，表示初始数列。保证 $|a_i| \le 10^6$。
接下来 q 行，每行一个操作，为以下两种之一：

- `1 i x`：给定 i, x，将 $a[i]$ 加上 x；
- `2 l r`：给定 l, r，求 $\sum_{i=l}^{r} a_i$ 的值。

保证 $1 \le l \le r \le n, |x| \le 10^6$。

输出

对于每个 `2 l r` 操作输出一行，每行有一个整数，表示所求的结果。

样例输入

```
3 2
1 2 3
1 2 0
2 1 3
```

样例输出

```
6
```

线段树的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

1. 定义线段树

创建一个结构体，代表每一个节点，每一个节点所含有的信息有l，r表示区间的范围，此外我们现在想要求和，那么就还需要有一个sum表示区间和，同时我们使用线段树来解决问题，一般需要开**四倍大小的空间**。

```
struct node{
    int l, r;
    LL sum;
}tr[N << 2];
```



线段树的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

2. 创建线段树

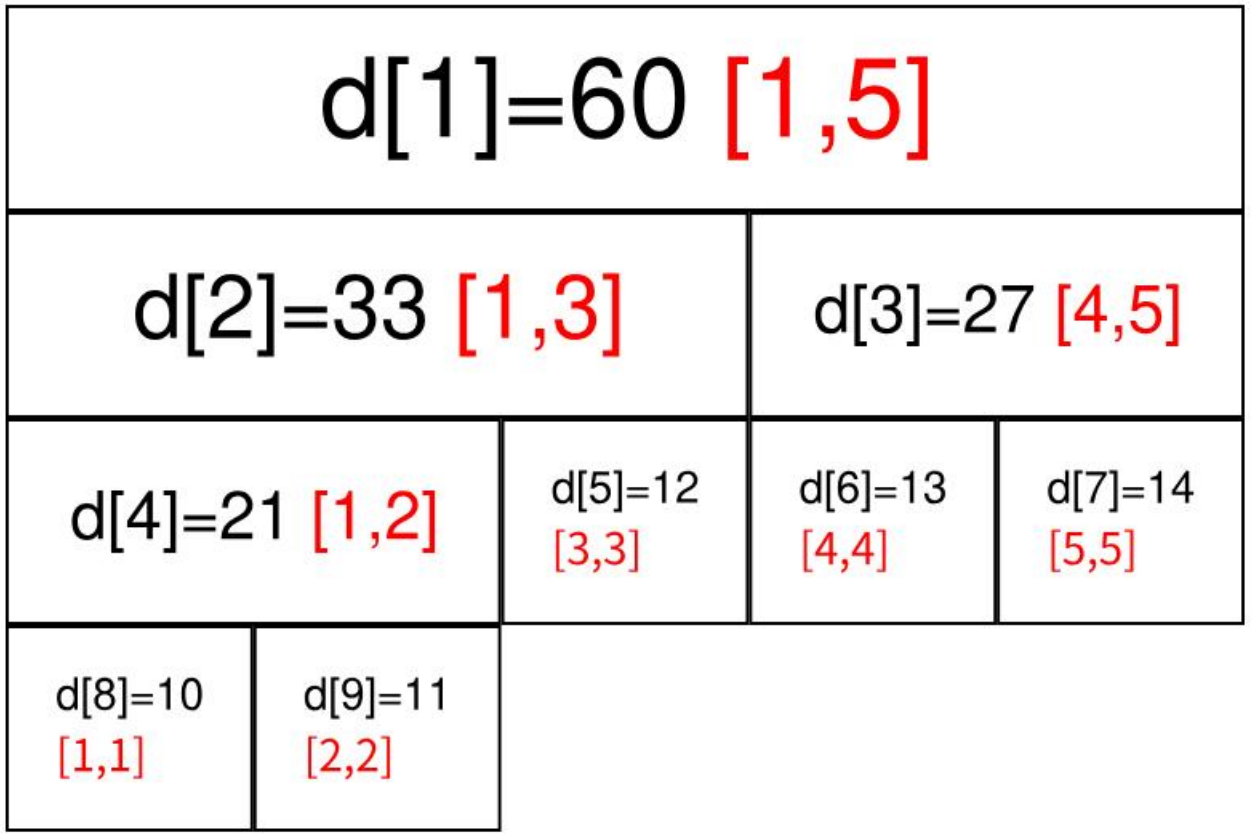
采用递归的方法创建线段树，算法步骤如下：

- 若是叶子节点（l == r），则节点的sum就是对应位置的元素值
- 若是非叶子节点，则递归创建左子树和右子树
- 节点的区间和等于该节点左右子树区间和的相加

```
void pushup(int u)
{
    tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum;
}
```

```
void build(int u, int l, int r)
{
    tr[u].l = l, tr[u].r = r;
    if(l == r)//叶子节点
    {
        tr[u].sum = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);//递归创建左右子树
    build(u << 1 | 1, mid + 1, r);
    pushup(u);//上传sum的值
}
```





线段树的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

3. 区间查询

查询一个[l , r]区间的区间和。采用递归的方法进行区间查询的算法步骤如下：

- 若节点所在的区间被查询区间[l , r]覆盖，则返回该节点的区间和
- 判断是在左子树中查询，还是在右子树中查询
- 返回区间和

```
LL query(int u, int l, int r)
{
    if(tr[u].l >= l && tr[u].r <= r) return tr[u].sum;
    int mid = (tr[u].l + tr[u].r) >> 1;
    LL res = 0;
    if(l <= mid) res += query(u << 1, l, r);
    if(r > mid) res += query(u << 1 | 1, l, r);
    return res;
}
```



线段树的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

4. 点更新

点更新指修改一个元素的值，例如将a [i]修改为v 。采用递归进行点更新，算法步骤如下：

- 若i属于当前节点所在区间，更新当前节点区间的区间和
- 若是非叶子节点，则判断是在左子树中更新还是在右子树中更新，是叶子节点，更新完毕，返回

```
void update(int u, int x, int v)
{
    if(tr[u].l == x && tr[u].r == x)
    {
        tr[u].sum = v;
        return;
    }
    int mid = (tr[u].l + tr[u].r) >> 1;
    if(x <= mid) update(u << 1, x, v);
    else update(u << 1 | 1, x, v);
    pushup(u);//修改完之后记得更新父节点

}
```



```cpp
void pushup(int u)
{
    tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum;
}
```

```cpp
void build(int u, int l, int r)
{
    tr[u].l = l, tr[u].r = r;
    if(l == r)//叶子节点
    {
        tr[u].sum = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(u << 1, l, mid);//递归创建左右子树
    build(u << 1 | 1, mid + 1, r);
    pushup(u);//上传sum的值
}
```

```cpp
void update(int u, int x, int v)
{
    if(tr[u].l == x && tr[u].r == x)
    {
        tr[u].sum += v;
        return;
    }
    int mid = (tr[u].l + tr[u].r) >> 1;
    if(x <= mid) update(u << 1, x, v);
    else update(u << 1 | 1, x, v);
    pushup(u);//修改完之后记得更新父节点
}
```

```cpp
LL query(int u, int l, int r)
{
    if(tr[u].l >= l && tr[u].r <= r) return tr[u].sum;
    int mid = (tr[u].l + tr[u].r) >> 1;
    LL res = 0;
    if(l <= mid) res += query(u << 1, l, r);
    if(r > mid) res += query(u << 1 | 1, l, r);
    return res;
}
```



```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N = 1e6 + 10;
struct node{
    int l, r;
    LL sum;
}tr[N << 2];
int a[N];
void pushup(int u)
{...}
void build(int u, int l, int r)
{...}
LL query(int u, int l, int r)
{...}
void update(int u, int x, int v)
{...}
int n, q;
int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    cin>>n>>q;
    for(int i = 1; i <= n; i ++) cin>>a[i];
    build(1, 1, n);
    while(q --)
    {
        int op, x, y;
        cin>>op>>x>>y;
        if(op == 1) update(1, x, y);
        else cout<<query(1, x, y)<<"\n";
    }
    return 0;
}
```

线段树的区间修改、区间查询

接下来我们来看一道关于线段树**区间修改**，**区间查询**的问题。



3495: 【模板】线段树区间修改、区间查询

时间限制: 1 Sec 内存限制: 128 MB
提交: 0 解决: 0
[提交] [状态] [讨论版] [命题人: 13305755511] [Edit] [TestData]

难度

★★★☆☆

题目描述

这是一道模板题。

给定数列 $a[1], a[2], \ldots, a[n]$，你需要依次进行 q 个操作，操作有两类：

- `1 l r x`：给定 l, r, x，对于所有 $i \in [l, r]$，将 $a[i]$ 加上 x（换言之，将 $a[l], a[l+1], \ldots, a[r]$ 分别加上 x）；
- `2 l r`：给定 l, r，求 $\sum_{i=l}^{r} a[i]$ 的值（换言之，求 $a[l] + a[l+1] + \cdots + a[r]$ 的值）。

输入

第一行包含 2 个正整数 n, q，表示数列长度和询问个数。保证 $1 \le n, q \le 10^6$。
第二行 n 个整数 $a[1], a[2], \ldots, a[n]$，表示初始数列。保证 $|a[i]| \le 10^6$。
接下来 q 行，每行一个操作，为以下两种之一：

- `1 l r x`：对于所有 $i \in [l, r]$，将 $a[i]$ 加上 x；
- `2 l r`：输出 $\sum_{i=l}^{r} a[i]$ 的值。

保证 $1 \le l \le r \le n, |x| \le 10^6$。

输出

对于每个 `2 l r` 操作，输出一行，每行有一个整数，表示所求的结果。

样例输入

```
5 10
2 6 6 1 1
2 1 4
1 2 5 10
2 1 3
2 2 3
1 2 2 8
1 2 3 7
1 4 4 10
2 1 2
1 4 5 6
2 3 4
```

样例输出

```
15
34
32
33
50
```

线段树的区间修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

1. 区间更新

- 那么对于区间修改的问题，我们之前已经在块状数组的时候学习到了一个技巧，我们可以利用一个「懒标记」来实现降低时间复杂度的目的。
- 在块状数组中如果修改的是一个完整的区间，那么我可以用块标记记录一下，当下次查询的时候，原本的值加上块标记的值才是真实的值。
- 在线段树中，懒标记也是同理，修改一个线段区间时，只对这个线段区间进行整体修改 (把修改记录在子树的根结点上)，其内部每个元素的内容先不做修改；当这个线段区间的一致性被破坏时，才把变化值传递给下一层。



线段树的区间修改、区间查询

COOLMAKER 酷哥创客 人工智能教育

那么这个时候，对于线段树的定义，需要在结构体中新增加一个lazy标记，用于记录累加的数是多少。

在查询操作的时候，我们会发现如果当前区间被分裂成两个区间了（一致性被破坏），需要先进行下传标记的操作，再进行递归查询。

在修改操作的时候，我们会发现现在修改的是一个区间了：

- 如果当前区间被完全包含在所要查询的区间之内，那么我们直接给这个区间的懒标记加上v，同时更新这个节点的sum即可。
- 如果当前区间被分裂成两个区间了（一致性被破坏），同样需要先进行下传标记的操作，再进行递归修改。

```
struct node{
    int l, r;
    LL sum, lazy;
}tr[N << 2];
```

```
LL query(int u, int l, int r)
{
    if(tr[u].l >= l && tr[u].r <= r) return tr[u].sum;
    pushdown(u);//分裂成两个区间了，需要先下传标记
    int mid = (tr[u].l + tr[u].r) >> 1;
    LL res = 0;
    if(l <= mid) res += query(u << 1, l, r);
    if(r > mid) res += query(u << 1 | 1, l, r);
    return res;
}
```

```
void update(int u, int l, int r, int v)
{
    if(tr[u].l >= l && tr[u].r <= r)
    {
        tr[u].lazy += v;
        tr[u].sum += (tr[u].r - tr[u].l + 1) * v;
        return;
    }
    pushdown(u);//分裂成两个区间了，需要先下传标记
    int mid = (tr[u].l + tr[u].r) >> 1;
    if(l <= mid) update(u << 1, l, r, v);
    if(r > mid) update(u << 1 | 1, l, r, v);
    pushup(u);//修改完之后记得更新父节点
}
```



线段树的区间修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

最后我们来看一下这个pushdown函数是怎么实现的。
首先我们要清楚，pushdown函数是用来把父节点的标记下传到子节点。如果当前节点已经是叶子结点，那么直接返回。如果当前节点的**lazy为0**，同样不需要操作。
其次，既然我们要下传lazy标记了，那么对于当前节点的两个儿子节点，需要更新他们的sum值（区间长度 乘 父节点的lazy）同时对于两个儿子还得继承父节点的lazy值，最后当前节点的lazy需要清空。

```
void pushdown(int u)
{
    if(tr[u].lazy)
    {
        tr[u << 1].sum += (tr[u << 1].r - tr[u << 1].l + 1LL) * tr[u].lazy;
        //左儿子的sum 加上区间长度*父节点的lazy
        tr[u << 1 | 1].sum += (tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1LL) * tr[u].lazy;
        tr[u << 1].lazy += tr[u].lazy;  //继承父节点的lazy
        tr[u << 1 | 1].lazy += tr[u].lazy;
        tr[u].lazy = 0; //清空父节点的lazy
    }
}
```



线段树的区间修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

先将 [1 , 4]区间内元素都加 10

区间	和	懒标记
[1 , 5]	55	0

区间	和	懒标记
[1 , 3]	(3-1+1)*10+6=36	10

区间	和	懒标记
[4 , 5]	19	0

[1 , 2]	3	0

[3 , 3]	3	0

[4 , 4]	14	10

[5 , 5]	5	0

[1 , 1]	1	0

[2 , 2]	2	0



线段树的区间修改、区间查询

查询区间[2 , 5]的和
12+13+19=44

区间	和	懒标记
[1 , 5]	55	0

区间	和	懒标记
[1 , 3]	36	0

先pushdown()

区间	和	懒标记
[1 , 2]	23	0

区间	和	懒标记
[3 , 3]	13	10

区间	和	懒标记
[4 , 5]	19	0

区间	和	懒标记
[4 , 4]	14	10

区间	和	懒标记
[5 , 5]	5	0

先pushdown()

区间	和	懒标记
[1 , 1]	11	10

区间	和	懒标记
[2 , 2]	12	10

线段树的区间修改、区间查询

接下来将区间[2，5]内元素加20



区间	和	懒标记
[1，5]	135	0

区间	和	懒标记
[1，3]	76	0

先pushdown()

区间	和	懒标记
[1，2]	43	0

区间	和	懒标记
[3，3]	33	30

区间	和	懒标记
[4，5]	59	20

区间	和	懒标记
[4，4]	14	10

区间	和	懒标记
[5，5]	5	0

先pushdown()

区间	和	懒标记
[1，1]	11	10

区间	和	懒标记
[2，2]	32	30

线段树的区间修改、区间查询

```
void pushdown(int u)
{
    if(tr[u].lazy)
    {
        tr[u << 1].sum += (tr[u << 1].r - tr[u << 1].l + 1LL) * tr[u].lazy;
        //左儿子的sum 加上区间长度*父节点的lazy
        tr[u << 1 | 1].sum += (tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1LL) * tr[u].lazy;
        tr[u << 1].lazy += tr[u].lazy;  //继承父节点的lazy
        tr[u << 1 | 1].lazy += tr[u].lazy;
        tr[u].lazy = 0; //清空父节点的lazy
    }
}
```

```
LL query(int u, int l, int r)
{
    if(tr[u].l >= l && tr[u].r <= r) return tr[u].sum;
    pushdown(u);//分裂成两个区间了，需要先下传标记
    int mid = (tr[u].l + tr[u].r) >> 1;
    LL res = 0;
    if(l <= mid) res += query(u << 1, l, r);
    if(r > mid) res += query(u << 1 | 1, l, r);
    return res;
}
```

```
void update(int u, int l, int r, int v)
{
    if(tr[u].l >= l && tr[u].r <= r)
    {
        tr[u].lazy += v;
        tr[u].sum += (tr[u].r - tr[u].l + 1LL) * v;
        return;
    }
    pushdown(u);//分裂成两个区间了，需要先下传标记
    int mid = (tr[u].l + tr[u].r) >> 1;
    if(l <= mid) update(u << 1, l, r, v);
    if(r > mid) update(u << 1 | 1, l, r, v);
    pushup(u);//修改完之后记得更新父节点
}
```

```
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N = 1e6 + 10;
struct node{
    int l, r;
    LL sum, lazy;
}tr[N << 2];
int a[N];
void pushup(int u)
{...}
void pushdown(int u)
{...}
void build(int u, int l, int r)
{...}
LL query(int u, int l, int r)
{...}
void update(int u, int l, int r, int v)
{...}
int n, q;
int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    cin>>n>>q;
    for(int i = 1; i <= n; i ++) cin>>a[i];
    build(1, 1, n);
    while(q --)
    {
        int op, x, y, z;
        cin>>op>>x>>y;
        if(op == 1)
        {
            cin>>z;
            update(1, x, y, z);
        }
        else cout<<query(1, x, y)<<"\n";
    }
    return 0;
}
```



扫描线

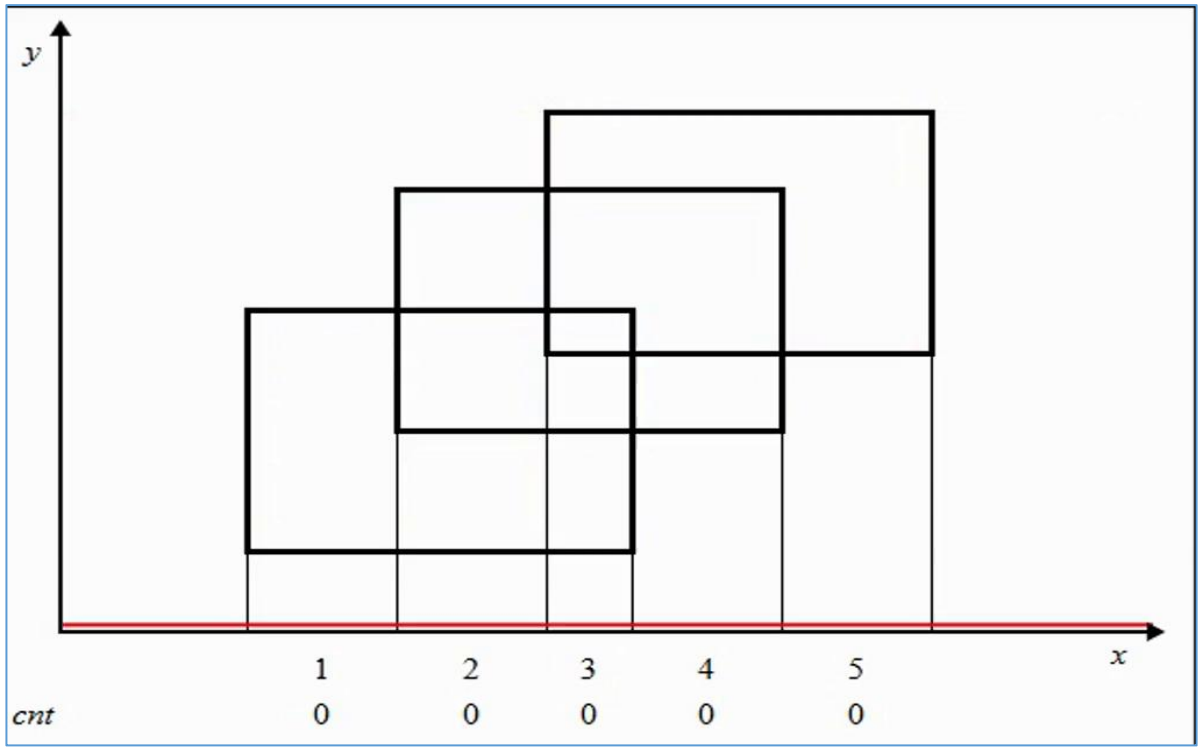




扫描线

扫描线一般运用在图形上面，它和它的字面意思十分相似，就是一条线在整个图上扫来扫去，它一般被用来解决图形面积，周长，以及二维数点等问题。

扫描线是一种思想，一般需要线段树或树状数组来实现

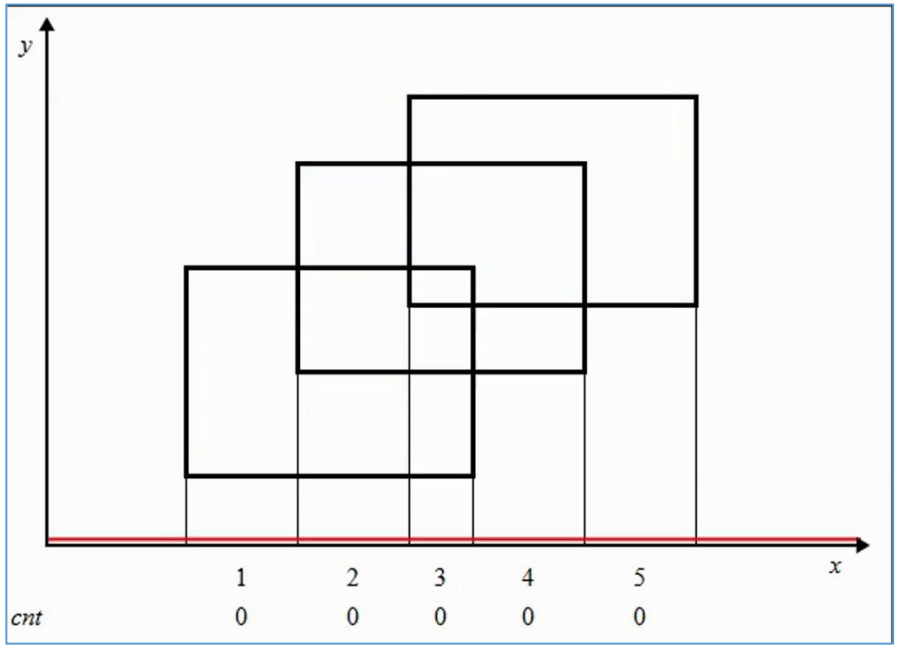


二维矩形面积并问题

COOLMAKER
酷哥创客 人工智能教育

在二维坐标系上，给出多个矩形的左下以及右上坐标，求出所有矩形构成的图形的面积。

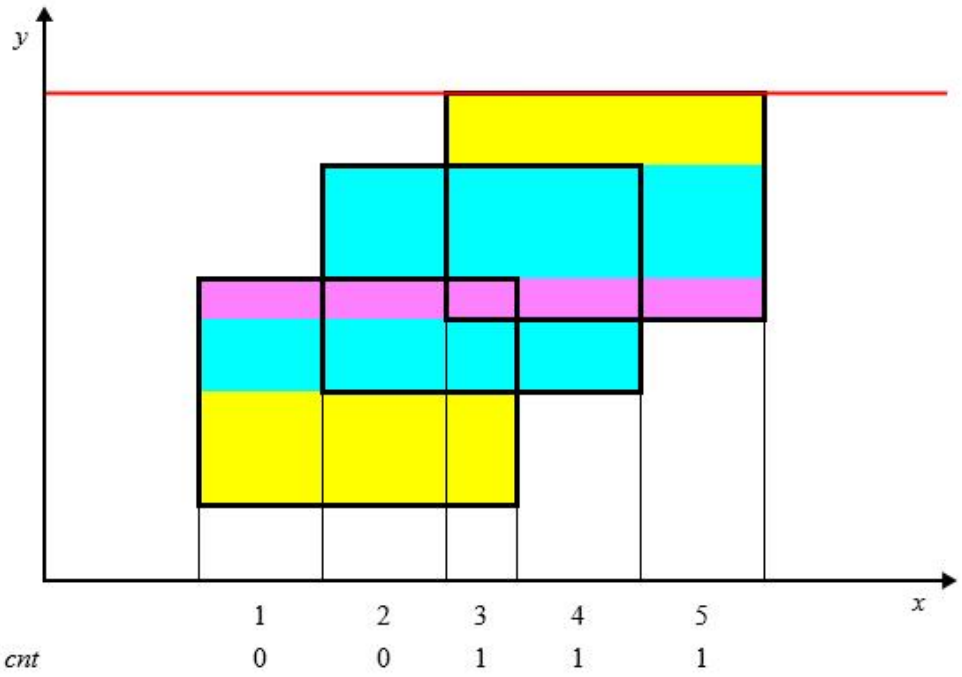
总面积可以直接暴力即可求出面积，如果数据大了怎么办？
这时就需要讲到 扫描线 算法。
现在假设我们有一根线，从下往上开始扫描：



二维矩形面积并问题

COOLMAKER 酷哥创客 人工智能教育

如图所示，把整个矩形分成如图各个颜色不同的小矩形，小矩形的高是扫过的距离，然而矩形的水平宽一直在变化。



给每一个矩形的上下边进行标记，下面的边标记为 1，上面的边标记为 -1。每遇到一个水平边时，让这条边（在横轴投影区间）的权值加上这条边的标记。

这个操作类似遍历括号序列：开括号加 1，闭括号减 1，「权值」对应当前位置的深度，「权值」是否大于 0，对应当前在不在括号里，也就是这段区间是否记入小矩形的宽度。

小矩形（不一定只有一个）的宽度就是整个数轴上权值大于 0 的区间总长度。

二维矩形面积并问题

COOLMAKER
酷哥创客 人工智能教育

用线段树维护矩形的长，也就是整个数轴上覆盖次数大于 0 的点。
需求列举如下：
1.一段区间权值加 1、减 1。
2.统计整个数轴上，区间权值大于 0 的「区间长度和」。

因为扫描线是从下往上扫描的，所以我们可以给下边赋上权值 1，给上边赋上权值 −1。当我们遇到下边时，说明这个矩阵还会在上面若干个部分出现，所以它对应的区间被覆盖次数加 1，之后统计它的影响。当我们遇到一条上边时，之前一定已经统计完了对应下边的影响，这时应该把对应区间的被覆盖次数 −1 ，代表这个图形的面积被统计完毕。

然而最后还有一个问题：线段树中存在对应长度为1的结点。这样的结点对应的是一个平面上的 x 坐标。显然一条线段的两个端点如果重合，它就等价于一个单点。这种叶结点会维护错误。

解决办法是令线段树中原本代表区间 [l,r]的结点代表区间 [l,r+1]。这样最小规模的结点也至少保存了原本的一条切割后的横边。



5486: 【模板】扫描线 & 矩形面积并

COOLMAKER
酷哥创客 人工智能教育

题目描述

求 n 个四边平行于坐标轴的矩形的面积并。

输入

第一行一个正整数 n。

接下来 n 行每行四个非负整数 x_1, y_1, x_2, y_2，表示一个矩形的四个端点坐标为 $(x_1, y_1), (x_1, y_2), (x_2, y_2), (x_2, y_1)$。

输出

一行一个正整数，表示 n 个矩形的并集覆盖的总面积。

样例输入1 复制

```
2
100 100 200 200
150 150 250 255
```

样例输出1 复制

```
18000
```

提示

对于 20% 的数据，$1 \le n \le 1000$。
对于 100% 的数据，$1 \le n \le 10^5$，$0 \le x_1 < x_2 \le 10^9$，$0 \le y_1 < y_2 \le 10^9$。

COOLMAKER
酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 3e5 + 5;
struct line
{...}
int n, all[N], tot, all2[N], tot2;
namespace seg
{...}
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n;
    for(int i = 1, x1, y1, x2, y2; i <= n; i++)
    {
        cin >> x1 >> y1 >> x2 >> y2;
        a[i * 2 - 1] = {y1, x1, x2, 1};
        a[i * 2] = {y2, x1, x2, -1};
        all[++tot] = x1;
        all[++tot] = x2;
        all2[++tot2] = y1;
        all2[++tot2] = y2;
    }
    sort(all + 1, all + tot + 1);
    tot = unique(all + 1, all + tot + 1) - all - 1;
    sort(all2 + 1, all2 + tot2 + 1);
    tot2 = unique(all2 + 1, all2 + tot2 + 1) - all2 - 1;
    sort(a + 1, a + 2 * n + 1);
    ll ans = 0;
    for(int i = 1, j = 1; i < tot2; i++)
    {
        for(; j <= 2 * n && all2[i] == a[j].t; j++)
        {
            a[j].l = lower_bound(all + 1, all + tot + 1, a[j].l) - all;
            a[j].r = lower_bound(all + 1, all + tot + 1, a[j].r) - all;
            seg::add(a[j].l, a[j].r - 1, a[j].val);
        }
        ans += 1ll * (all2[i + 1] - all2[i]) * seg::t[1];
    }
    cout << ans << "\n";
    return 0;
}
```

```cpp
struct line
{
    int t, l, r, val;
    bool operator <(const line &a)const
    {
        return t < a.t;
    }
} a[N];
int n, all[N], tot;
struct seg
{
    int t[N << 2], lazy[N << 2];
    void add(int L, int R, int v, int rt = 1, int l = 1, int r = tot)
    {
        if(L <= l && r <= R)
        {
            lazy[rt] += v;
            if(lazy[rt] > 0) t[rt] = all[r + 1] - all[l];
            else t[rt] = t[rt * 2] + t[rt * 2 + 1];
            return ;
        }
        int mid = (l + r) >> 1;
        if(L <= mid) add(L, R, v, rt << 1, l, mid);
        if(mid < R)  add(L, R, v, rt << 1 | 1, mid + 1, r);
        if(lazy[rt] > 0) t[rt] = all[r + 1] - all[l];
        else t[rt] = t[rt * 2] + t[rt * 2 + 1];
    }
} tree;
```

线与对应线会抵消，采用分治的思想，lazy相当于不用下放的懒标记

5005: 二元环

COOLMAKER
酷哥创客 人工智能教育

题目描述

小酷宝在魔法学院学习图论时，设计了一个有趣的二分图模型：

1. 图中有两个不相交的顶点集：左集 L （n 个顶点，编号 1 到 n）和右集 R （m 个顶点，编号 1 到 m）
2. 对于每个左顶点 $i \in L$，它会向右顶点区间 $[l_i, r_i]$ 中的所有顶点连一条有向边（$i \to j$，其中 $j \in [l_i, r_i]$）
3. 对于每个右顶点 $j \in R$，它会向左顶点区间 $[L_j, R_j]$ 中的所有顶点连一条有向边（$j \to i$，其中 $i \in [L_j, R_j]$）

现在小酷宝想知道，这个图中存在多少个"二元环"(即满足 $i \to j$ 和 $j \to i$ 同时存在的顶点对 (i, j))。

输入

第一行 2 个数 n, m，意义在题目描述中已经给出；

接下来 n 行，每行 2 个数 l, r ($1 \le l \le r \le m$)，表示左边对应点向右边区间连边；

接下来 m 行，每行 2 个数 L, R ($1 \le L \le R \le n$)，表示右边对应点向左边区间连边。

对于所有数据，保证 $1 \le n, m \le 10^6$ 。

对于 10% 数据，满足 $1 \le n, m \le 10$ 。

对于另外 10% 数据，满足 $1 \le n, m \le 2000$ 。

对于另外 20% 数据，满足 $1 \le n, m \le 2 \times 10^5$ 。

对于 100% 数据，保证 $1 \le n, m \le 10^6$ 。

输出

一行一个数，为答案。

样例输入1 复制

```
3 3
2 3
3 3
1 3
2 3
2 2
1 1
```

考虑先存下第一种从左边连向右边的点。然后考虑扫描右边的点，用线段树或树状数组维护有那些左边的点连向右边，然后直接查询该点的左部点的区间有多少点连过来即可。
所用数据结构支持单点修改和区间查询即可

```cpp
#include<bits/stdc++.h>
using namespace std;
#define ll long long
const int N = 1e6 + 10;
ll tree[4 * N];
vector<pair<int, int>> e[N];
void push_up(int l, int r, int p)
{...}
void update(int l, int r, int p, int x, int y, ll val)
{...}
ll query(int l, int r, int p, int x, int y)
{...}
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    int n, m;
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
    {
        int l, r;
        cin >> l >> r;
        e[l].push_back({i, 1});
        e[r + 1].push_back({i, -1});
    }
    int maxn = max(n, m);
    ll ans = 0;
    for(int i = 1; i <= m; i++)
    {
        for(auto now : e[i])
        {
            update(1, maxn, 1, now.first, now.first, now.second);
        }
        int l, r;
        cin >> l >> r;
        ans += query(1, maxn, 1, l, r);
    }
    cout << ans << "\n";
}
```



考虑先存下第一种从左边连向右边的点。然后考虑扫描右边的点，用线段树或树状数组维护有那些左边的点连向右边，然后直接查询该点的左部点的区间有多少点连过来即可。
所用数据结构支持单点修改和区间查询即可

```
void push_up(int l, int r, int p)
{
    tree[p] = tree[p * 2] + tree[p * 2 + 1];
}
void update(int l, int r, int p, int x, int y, ll val)
{
    if(x <= l && r <= y)
    {
        tree[p] += val;
        return;
    }
    int mid = (l + r) / 2;
    if(x <= mid)
    {
        update(l, mid, p * 2, x, y, val);
    }
    if(mid < y)
    {
        update(mid + 1, r, p * 2 + 1, x, y, val);
    }
    push_up(l, r, p);
}
ll query(int l, int r, int p, int x, int y)
{
    if(x <= l && r <= y)
    {
        return tree[p];
    }
    int mid = (l + r) / 2;
    ll ans = 0;
    if(x <= mid)ans += query(l, mid, p * 2, x, y);
    if(mid < y)ans += query(mid + 1, r, p * 2 + 1, x, y);
    return ans;
}
```

酷哥创客

感谢您的观看





对顶栈

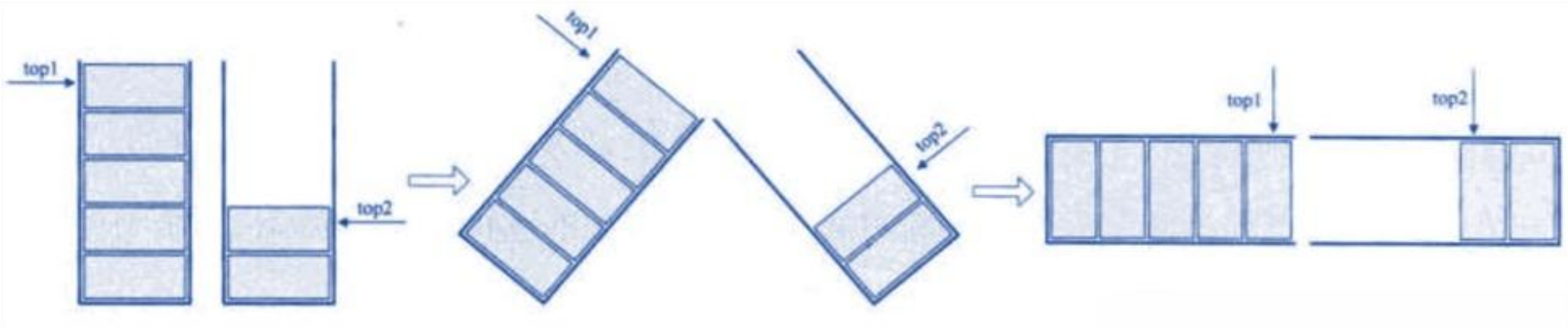




什么是双端栈?

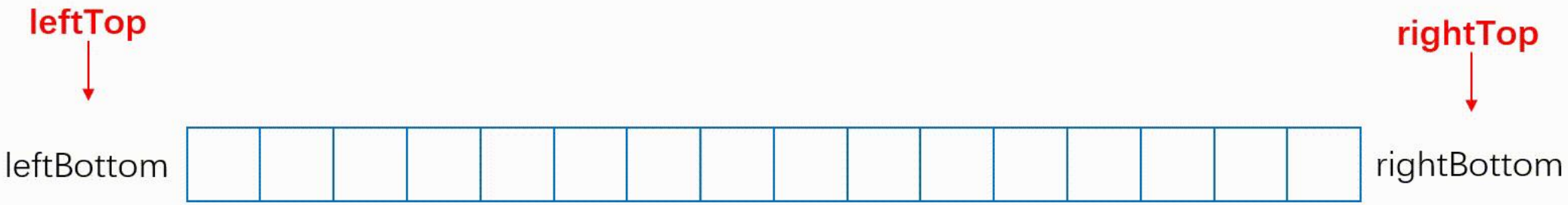
双端栈是指将一个线性表的两端当做栈底分别进行入栈和出栈操作，本质上像是两个栈共享同一块存储空间，一个从数组头开始生长，另一个从数组尾开始生长，向中间汇合。

特别地，对于两个栈的使用，可以让它们共用一个数组空间，采用栈顶相向、迎面增长的存储方式，这就是双端栈，又叫对顶栈。

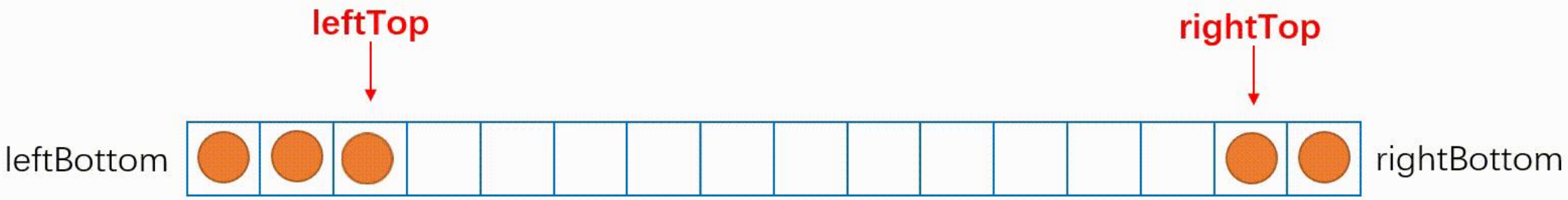


双端栈的顺序存储结构——入栈操作O(1)





双端栈的顺序存储结构——弹栈操作

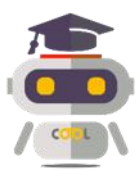


```c
#define MAXN 1005
int stack[MAXN];
int top1 = -1, top2 = MAXN; //初始化双端栈两个栈顶
//入栈
int push1(int x)
{
    if(top1 + 1 < top2)
    {
        stack[++top1] = x; //左栈栈顶右移，为++
        return 1;
    }
    return 0; //如果top1和top2相遇了，就无法添加
}
int push2(int x)
{
    if(top2 - 1 > top1)
    {
        stack[--top2] = x; //右栈栈顶是向左移，所以要--
        return 1;
    }
    return 0; //同push1
}
```

```c
//出栈
void pop1()
{
    if(top1 > -1) //栈不为空
    {
        top1--; //左栈pop元素，栈顶左移，对应--
    }
}
void pop2()
{
    if(top2 < MAXN) //栈不为空
    {
        top2++; //右栈pop元素，栈顶要右移，对应++
    }
}
```

```c
//判断栈是否为空
int empty1()
{
    if(top1 > -1)
    {
        return 0;
    }
    return 1;
}
int empty2()
{
    if(top2 < MAXN)
    {
        return 0;
    }
    return 1;
}
```

```c
//获取栈顶元素
int top1()
{
    if(!empty1())
    {
        return stack[top1];
    }
}
int top2()
{
    if(!empty2())
    {
        return stack[top2];
    }
}
//数组是否已满
int isFull()
{
    if(top1 + 1 == top2)
    {
        return 1;
    }
    return 0;
}
```

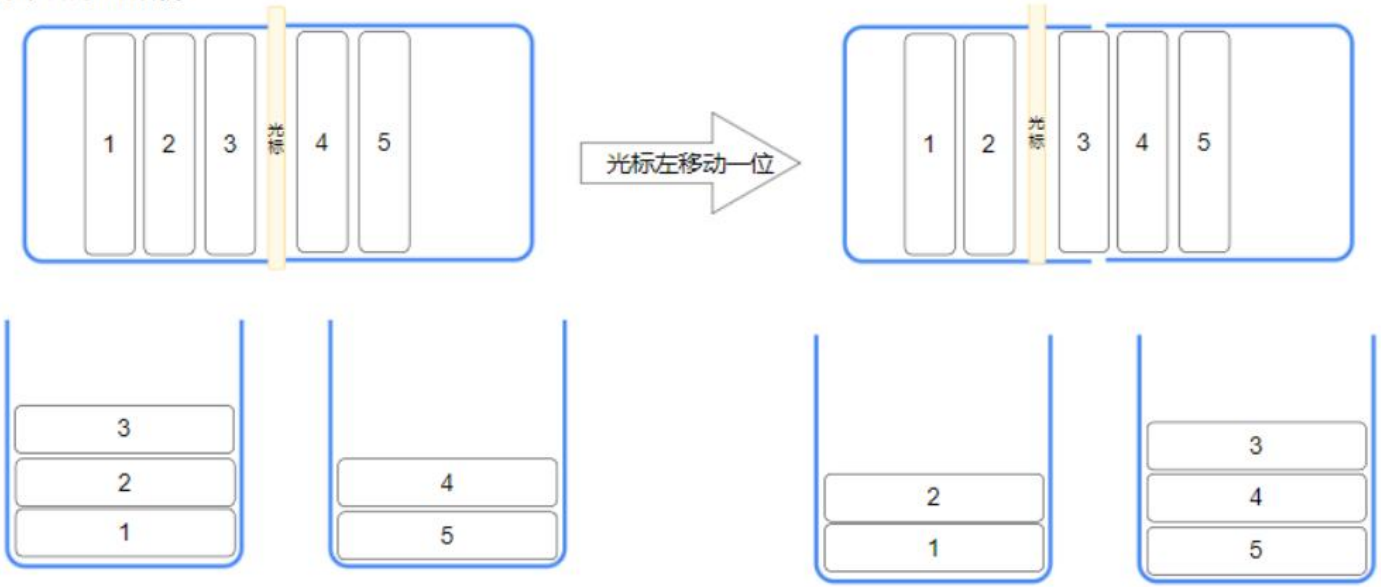


4847: 梦中数据结构

COOLMAKER 酷哥创客 人工智能教育

题目描述

在一个宁静的夜晚，小酷宝进入了梦乡。在他的梦中，他想到了一种问题：
在打字过程中，光标一次只能移动一位，在**光标前进行添加、删除数字**，每次想要知道光标前数字的一些信息，这样的信息可以使用**两个栈**去进行维护（分别表示光标前的数字序列，光标后的数字序列），每次左移，相当于将第一个栈的栈顶元素移动到第二个栈的栈顶。小酷宝非常高兴，将这种数据结构命名为“**对顶栈**”。



梦醒了，现在，小酷宝想把这个梦中的数据结构变成现实，编写一个“数列编辑器”。
一开始，**数列为空，光标在开头位置**，编辑器能进行如下5种操作:

`I x` ：在光标的后面**插入**一个数 `x` ,并把光标移到新加入的 `x` 后；
`D` ：**删除光标前**的最后一个位置的数（保证存在），光标**相对位置**不变；
`L` ：光标**左移**一位，如果已经在开头则不用做任何事；
`R` ：光标**右移**一位，如果已经在结尾不用做任何事；
`Q l r` ：编辑器需要给出 $A[l], A[l+1], A[l+2] \cdots A[r] (l \leq r)$ 的**和**，**保证** $l \leq r \leq n$ ， n 为当前光标前的数字个数（保证存在至少一个数）。

```cpp
#include <bits/stdc++.h>
using namespace std;
int q, x, l, r;
char op;
stack<int>Left, Right;
int sum[500005];
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> q;
    while(q--)
    {
        cin >> op;
        if(op == 'I')
        {
            cin >> x;
            Left.push(x);
            int n = Left.size();
            sum[n] = sum[n - 1] + Left.top();
        }
        else if(op == 'D')
        {
            Left.pop();
        }
        else if(op == 'L')
        {
            if(!Left.empty())
            {
                Right.push(Left.top());
                Left.pop();
            }
        }
        else if(op == 'R')
        {
            if(!Right.empty())
            {
                Left.push(Right.top());
                Right.pop();
                int n = Left.size();
                sum[n] = sum[n - 1] + Left.top();
            }
        }
        else
        {
            cin >> l >> r;
            cout << sum[r] - sum[l - 1] << "\n";
        }
    }
    return 0;
}
```

```cpp
#include<bits/stdc++.h>
using namespace std;
int stk[500005],sum[500005];
int top1 = 0, top2 = 500000, n, x, l, r;
char ch;
int main()
{
    ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    cin >> n;
    for(int i = 0; i < n; i++)
    {
        cin >> ch;
        if(ch == 'I') //I对应入栈
        {
            cin >> x;
            stk[++top1] = x;
            sum[top1] = sum[top1 - 1] + stk[top1];
        }
        if(ch == 'D') //D对应出栈
        {
            if(top1 > 0) top1--;
        }
        if(ch == 'L') //L弹出左栈并入右栈
        {
            if(top1 > 0)
            {
                int tmp = stk[top1];
                top1--;
                stk[--top2] = tmp;
            }
        }
        if(ch == 'R') //R弹出右栈并入左栈
        {
            if(top2 < 500000)
            {
                int tmp = stk[top2];
                top2++;
                stk[++top1] = tmp;
                sum[top1] = sum[top1 - 1] + stk[top1];
            }
        }
        if(ch == 'Q') //Q询问并输出
        {
            cin >> l >> r;
            cout << sum[r] - sum[l - 1] << "\n";
        }
    }
    return 0;
}
```

COOLMAKER
酷哥创客 人工智能教育



单调栈与单调队列





什么是单调栈?

顾名思义，单调栈即满足单调性的栈结构，栈内元素要么从小到大，要么从大到小。

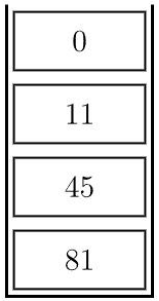
单调栈维护的就是一个数 前/后 第一个 大于/小于 它的数。



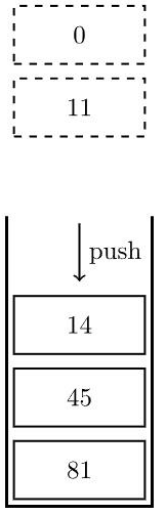
如何维护单调栈

为了描述方便，以下举例维护一个整数的**从栈顶到栈底单调递增**的栈为例。

例如，栈中自顶向下的元素为 $\{0, 11, 45, 81\}$。



插入元素 14 时为了保证单调性需要依次弹出元素 $0, 11$，操作后栈变为 $\{14, 45, 81\}$。




```
//把小于等于当前位置a[i]的元素弹出，最后栈顶就是第一个大于x的数了
//为了保证结构干净，栈中一般存的是对应值的下标
while(!st.empty()&&a[st.top()]<=a[i])
{
    st.pop();
}
st.push(i);
```

单调栈的应用

以O(N)的时间复杂度求出整个数组每个位置的左边或右边第一个比它大或小的元素。

什么时候使用单调栈：

给定一个长度为N的整数序列

1.输出**每个数**左边第一个比它小的数(或者求位置)

做法：从左到右遍历序列，当栈顶值大于等于当前位置的值，弹出，最后栈顶为目标值

2.输出**每个数**左边第一个比它大的数(或者求位置)

做法：从左到右遍历序列，当栈顶值小于等于当前位置的值，弹出，最后栈顶为目标值

3.输出**每个数**右边第一个比它小的数(或者求位置)

做法1：从右到左遍历序列，当栈顶值大于等于当前位置的值，弹出，最后栈顶为目标值

做法2：从左到右遍历序列，当栈顶值大于当前位置的值，弹出，栈顶位置的目标值就是当前位置

4.输出**每个数**右边第一个比它大的数(或者求位置)

做法1：从右到左遍历序列，当栈顶值小于等于当前位置的值，弹出，最后栈顶为目标值

做法2：从左到右遍历序列，当栈顶值小于当前位置的值，弹出，栈顶位置的目标值就是当前位置



3296: 【模板】单调栈

COOLMAKER
酷哥创客 人工智能教育

题目描述

给出项数为 n 的整数数列 $a_1, a_2, a_3, \cdots, a_n$ 。

定义函数 $f(i)$ 代表数列中第 i 个元素之后第一个大于 a_i 的元素的下标。若不存在，则 $f(i) = 0$ 。

试求出 $f(1), f(2), f(3), \cdots, f(n)$ 。

输入

第一行一个正整数 n 。

第二行 n 个正整数 a_i 。

【数据规模与约定】

对于 30% 的数据，$n \leq 100$;

对于 60% 的数据，$n \leq 5 \times 10^3$;

对于 100% 的数据，$1 \leq n \leq 3 \times 10^6, 1 \leq a_i \leq 10^9$ 。

输出

一行 n 个整数表示 $f(1), f(2), f(3), \cdots, f(n)$ 。的值。

样例输入1 复制

```
5
1 4 2 3 5
```

样例输出1 复制

```
2 5 4 5 0
```

序列	1	4	2	3	5
ans	2	5	4	5	0

4.输出每个数右边边第一个比它大的数(或者求位置)
做法1：从右到左遍历序列，当栈顶值小于等于当前位置的值，弹出，最后栈顶为目标值
做法2：从左到右遍历序列，当栈顶值小于当前位置的值，弹出，栈顶位置的目标值就是当前位置

COOLMAKER 酷哥创客 人工智能教育

```cpp
#include <bits/stdc++.h>
using namespace std;
int n;
int a[3000005];
int ans[3000005];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        cin>>a[i];
    }
    stack<int>st;
    for(int i=n;i>=1;i--)
    {
        //把小于等于当前位置a[i]的元素弹出，最后栈顶就是第一个大于x的数了
        //为了保证结构干净，栈中一般存的是对应值的下标
        while(!st.empty()&&a[st.top()]<=a[i])
        {
            st.pop();
        }
        if(!st.empty())ans[i]=st.top();
        st.push(i);
    }
    for(int i=1;i<=n;i++)
    {
        cout<<ans[i]<<" ";
    }
    return 0;
}
```

从右到左遍历序列

当栈顶值小于等于当前位置的值，弹出

最后栈顶为目标值



单调栈



序列	1	4	2	3	5
ans	2	5	4	5	0

4.输出每个数右边边第一个比它大的数(或者求位置)
做法1：从右到左遍历序列，当栈顶值小于等于当前位置的值，弹出，最后栈顶为目标值
做法2：从左到右遍历序列，当栈顶值小于当前位置的值，弹出，栈顶位置的目标值就是当前位置

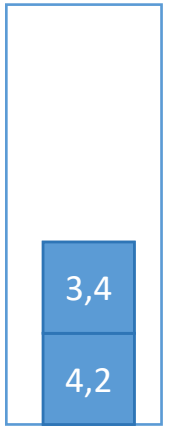


COOLMAKER 酷哥创客 人工智能教育

从左到右遍历序列

当栈顶值小于当前位置的值，弹出

```cpp
#include <bits/stdc++.h>
using namespace std;
int n;
int a[3000005];
int ans[3000005];
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        cin>>a[i];
    }
    stack<int>st;
    for(int i=1;i<=n;i++)
    {
        //把小于当前位置a[i]的元素弹出
        //每次弹出位置的目标值就是当前位置
        //为了保证结构干净，栈中一般存的是对应值的下标
        while(!st.empty()&&a[st.top()]<a[i])
        {
            ans[st.top()]=i;st.pop();
        }
        st.push(i);
    }
    for(int i=1;i<=n;i++)
    {
        cout<<ans[i]<<" ";
    }
    return 0;
}
```



单调栈

栈顶位置的目标值就是当前位置



题目描述

在云端学园的魔法图书馆里，收藏着一条由 N 颗彩虹水晶串成的项链。每颗水晶都刻着独一无二的数字：$1, 2, \cdots, N$，但顺序被调皮的风精灵打乱了。

小酷宝是学园里最年轻的魔法学徒，他需要用魔法棒在这条“数字彩虹”上画出一个**最漂亮的连续子序列——定义“漂亮值”**：子序列中**最小数字**。
每画一段子序列，就能获得“漂亮值”对应的魔法能量。，小酷宝想**收集所有可能的子序列的漂亮值之和！**帮小酷宝出：所有连续子序列的“漂亮值”总和是多少？

输入

第一行：一个整数 $N(1 \leq N \leq 200000)$（彩虹水晶的数量）。

第二行：N 个整数 $a_1, a_2, \cdots, a_n$，表示被打乱的 1~N 的排列。

输出

输出一行，一个整数：所有连续子序列的“最小值”之和。

样例输入1 复制

```
3
2 1 3
```

样例输出1 复制

```
9
```

样例输入2 复制

```
4
1 3 2 4
```

样例输出2 复制

```
19
```

```cpp
#include <bits/stdc++.h>
using namespace std;
int n,a[200005],l[200005],r[200005];
stack<int> s1,s2;
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i], l[i]=0, r[i]=n+1;
    long long  ans = 0;
    for(int i = 1; i <= n; i++)
    {
        while(!s1.empty() && a[s1.top()]>=a[i]) s1.pop();
        if(!s1.empty()) l[i]=s1.top();
        s1.push(i);
        while(!s2.empty() && a[s2.top()]>a[i])
        {
            r[s2.top()]=i; s2.pop();
        }
        s2.push(i);
    }
    for(int i = 1; i <= n; i++)
    {
        ans += 1ll * (i - l[i]) * (r[i] - i) * a[i];
    }
    cout << ans << endl;
    return 0;
}
```

什么是单调队列?

顾名思义，单调队列的重点分为「单调」和「队列」。

「单调」指的是元素的「规律」——递增（或递减）。

「队列」指的是元素只能从**队头**和**队尾**进行操作的双端队列。

单调队列是一种主要用于解决滑动窗口类问题的数据结构，即，在长度为 n 的序列中，求每个长度为 m 的区间的**区间最值**。它的时间复杂度是 O(n)，在这个问题中比O(nlog n)的**ST表和线段树**，以及O(nlog n log n)的**树状数组**要优。



单调队列原理

我们可以反向思考，把现在不是最美味的，将来也不可能成为最美的苹果从队列中删除，从而减少查询次数。

小酷宝参加一场真人秀，节目共录[illegible]
酷宝每一天只会吃天数不超过M(1<=M<=N<=1e6)[illegible]苹果。请问对于每一天，当前天能够吃的最美味苹果是哪一天的?
比如 N=5，M=4，5天提供的苹果美味度分别为（80，75，78，73，79），老徐做法如下：

暴力做法







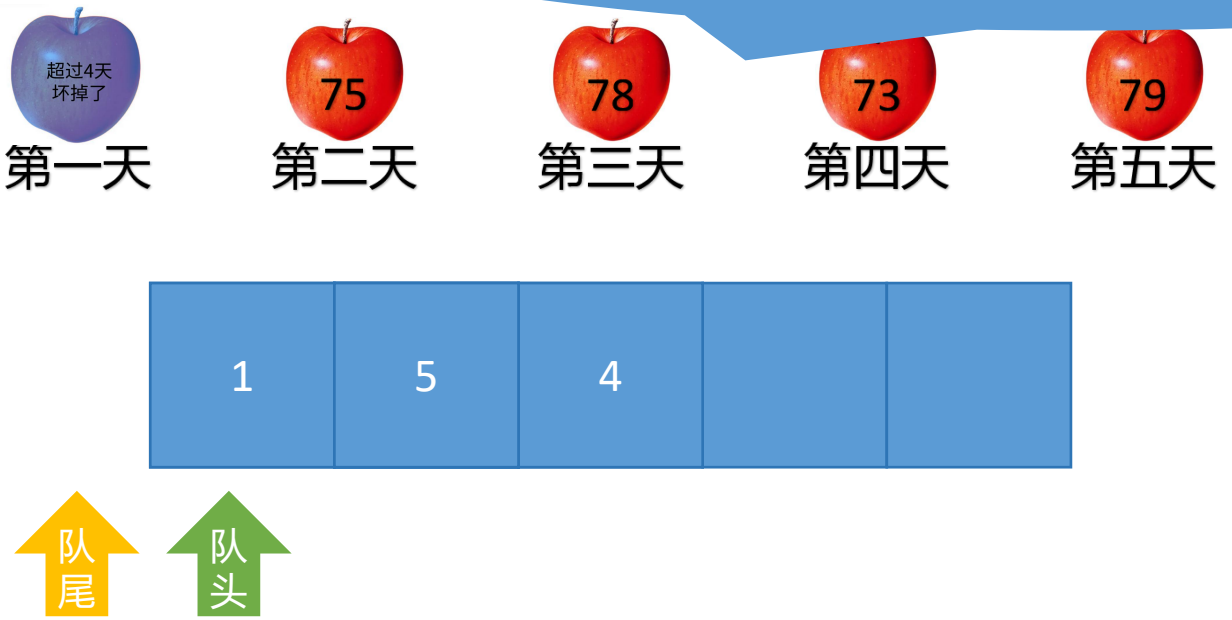
COOLMAKER
酷哥创客 人工智能教育

单调队列原理

75<78,并且第三天的苹果既美味又新鲜，所以
第二天的苹果不可能成为后面几天苹果中最美

在单调队列中，每一个苹果都进队一次，最多也只出队一次，而且每次查询只需取队头的苹果，所以整体的时间复杂度是O(n)

单调队列做法



单调队列的作用

单调队列常用来维护给定区间中的最值问题，其时间复杂度为O(n)，还有可以拿来优化DP
单调队列：通俗的讲就是以一个固定长度的窗口在数列上移动，（这个固定的长度是题目给的，队首元素在队列中的位置与队尾元素在队列的位置之差若大于这个固定长度，就得将队首元素弹出）

1.单调队列必须满足从队头到队尾的严格单调性

2.单调递增队列：队列中的元素从小到大排列；保证队头元素是当前队列的最小值，用于维护区间最小值
单调递减队列：队列中的元素从大到小排列；保证队头元素是当前队列的最大值，用于维护区间最大值

3.单调队列需要完成的三个基本操作：
去尾：去掉尾部元素（目的：维护队列单调性，新元素入队时，去掉尾部不满足单调的尾部元素）
掐头：掐掉头部元素（条件：当前队头元素所在区间不在目标区间范围，目的：维护目标区间）
取最值：取出队列的头元素，即为最值



单调队列原理

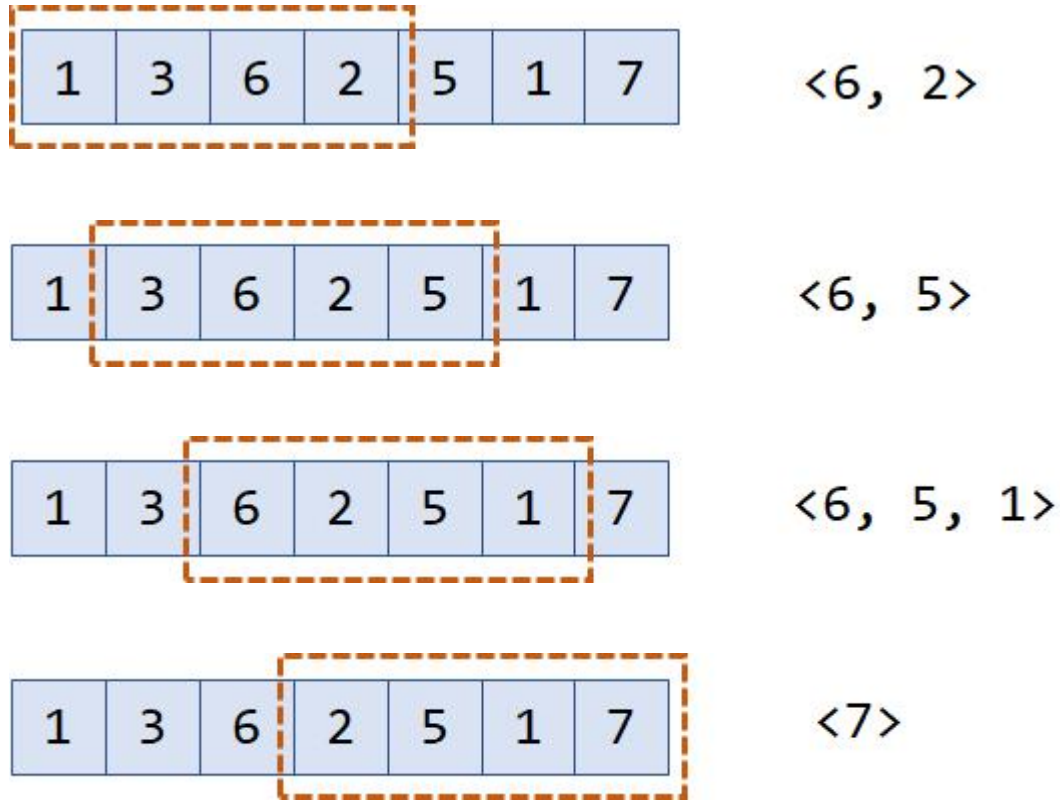
原序列	1	3	6	2	5	1	7

现维护一个单调递减的双向队列，要求的是每连续的 4 个数中的最大值。

去尾：当一个数进入所要 "寻找" 最大值的范围中时，若这个数比其前面（先进队）的数要大，显然，前面的数会比这个数先出队且不再可能是最大值，可将前面的数 "弹出"，再将该数真正 push 进队尾。

为了让单调队列的数据足够干净，在单调队列中，一般存储 原序列的下标 即可，而不需要存储原序列的值，根据保序性，存储的下标一定是单调递增的。

掐头：当队列中的队头下标小于了目前要查询区间的边界，"弹出"队头



2744: 滑动窗口/【模板】单调队列

时间限制: 2 Sec 内

提交: 0 解

[提交] [状态] [讨论版] [命题

难度

★★★★☆

题目描述

有一个长为n的序列a，以及一个大小为k的窗口。现

个单位，求出每次滑动后窗口中的最大值和最小值

该数组为 `[1 3 -1 -3 5 3 6 7]`，k为

窗口位置	最小值	最大值
[1 3 -1] -3 5 3 6 7	-1	3
1 [3 -1 -3] 5 3 6 7	-3	3
1 3 [-1 -3 5] 3 6 7	-3	5
1 3 -1 [-3 5 3] 6 7	-3	5
1 3 -1 -3 [5 3 6] 7	3	6
1 3 -1 -3 5 [3 6 7]	3	7

输入

输入一共两行，第一行有两个整数n,k。第二行n个

本题读入较多，请使用较快的输入输出技巧

输出

输出共两行，第一行是每次窗口滑动的最小值。

第二行是每次窗口滑动的最大值。

样例输入

```
8 3
1 3 -1 -3 5 3 6 7
```

样例输出

```
-1 -3 -3 -3 3 3
3 3 5 5 6 7
```

提示

对于 50% 的数据，$1 \leq n \leq 10^5$。

对于 100% 的数据，$1 \leq k \leq n \leq 10^6$，$a_i \in [-2^{31}, 2^{31})$。

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N=1e6+10;
int a[N],mi[N],ma[N];
deque<int>q1;
deque<int>q2;
int n,k;
int main()
{
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>k;
    for(int i=1; i<=n; i++)
    {
        cin>>a[i];
        //掐头
        while(!q1.empty()&&i-k>=q1.front()) q1.pop_front();
        while(!q2.empty()&&i-k>=q2.front()) q2.pop_front();
        //去尾
        while(!q1.empty()&&a[q1.back()]>a[i]) q1.pop_back();//保持升序-维护区间最小
        while(!q2.empty()&&a[q2.back()]<a[i]) q2.pop_back();//降-维护区间最大
        q1.push_back(i);
        q2.push_back(i);
        if(i>=k)//窗口大小已经为K了
        {
            mi[i]=a[q1.front()];//记录最小值
            ma[i]=a[q2.front()];//记录最大值
        }
    }
    for(int i=k; i<=n; i++)
    {
        cout<<mi[i]<<" ";
    }
    cout<<"\n";
    for(int i=k; i<=n; i++)
    {
        cout<<ma[i]<<" ";
    }
    cout<<"\n";
    return 0;
}
```



树状数组





树状数组

COOLMAKER
酷哥创客 人工智能教育

树状数组（Binary Index Tree, **BIT**）是最简洁优美的数据结构之一，是一种用于高效处理对一个存储数字的**列表**进行更新及求**前缀和**的数据结构；最简单的树状数组支持两种操作，时间复杂度均为O(log n)：

单点修改：更改数组中一个元素的值
区间查询：查询一个**区间内所有元素的和**

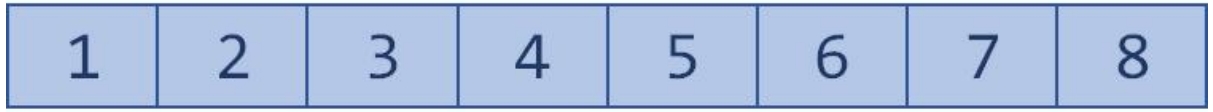


树状数组的引入

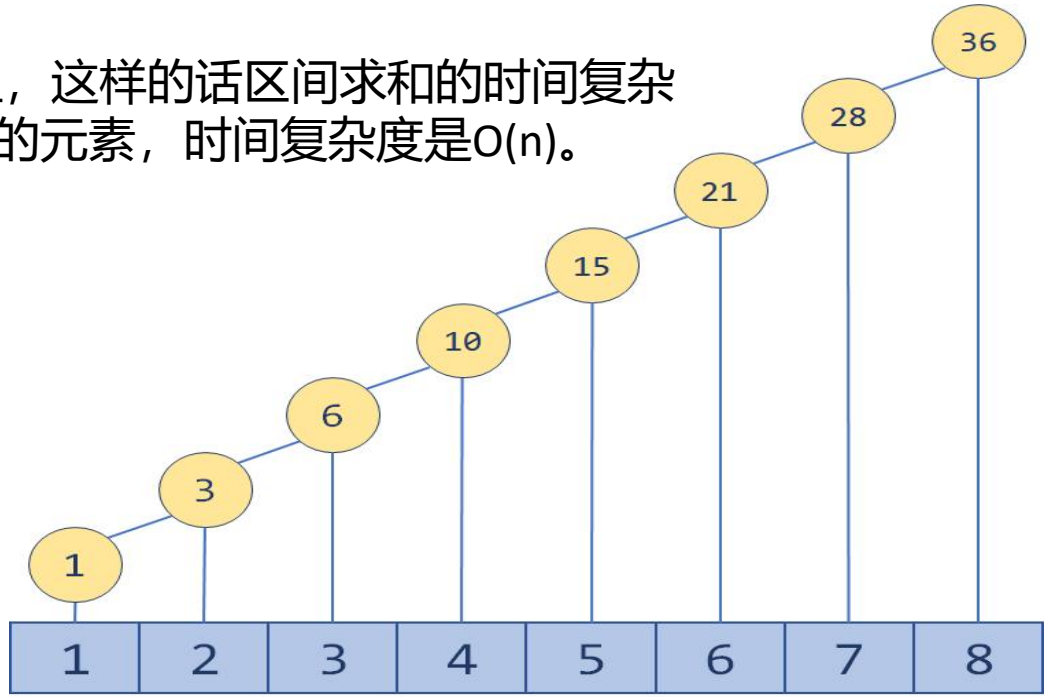
COOLMAKER
酷哥创客 人工智能教育

我们要实现两种操作：单点修改和区间求和。
对于普通数组而言，单点修改的时间复杂度是 O(1)，但区间求和的时间复杂度是 O(n)

当然，我们也可以用前缀和的方法维护这个数组，这样的话区间求和的时间复杂度就降到了O(1)，但是**单点修改**会影响后面所有的元素，时间复杂度是O(n)。



普通数组（蓝色）



前缀和数组（黄色）

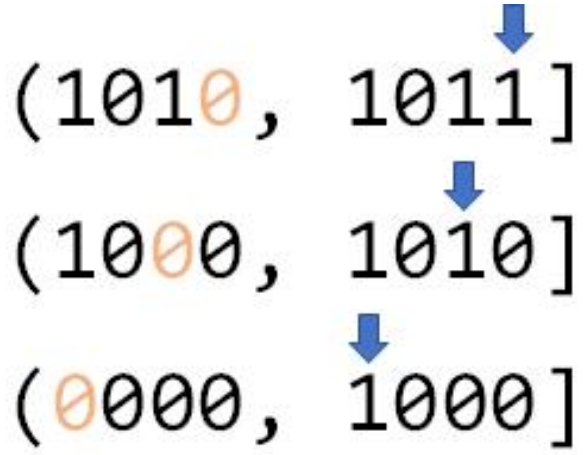
树状数组的引入

COOLMAKER
酷哥创客 人工智能教育

- 一个程序最后跑多长时间，是由最慢的一环决定的，所以上述两种方法的时间复杂度都是O(n)的。
- 因此现在我们希望找到这样一种折中的方法：无论单点修改还是区间查询，它都能不那么慢地完成。
- 树状数组就是这样一种结构，它巧妙地利用了二进制。

例如11，转化为二进制数就是$(1011)_2=2^3+2^1+2^0=8+2+1$。

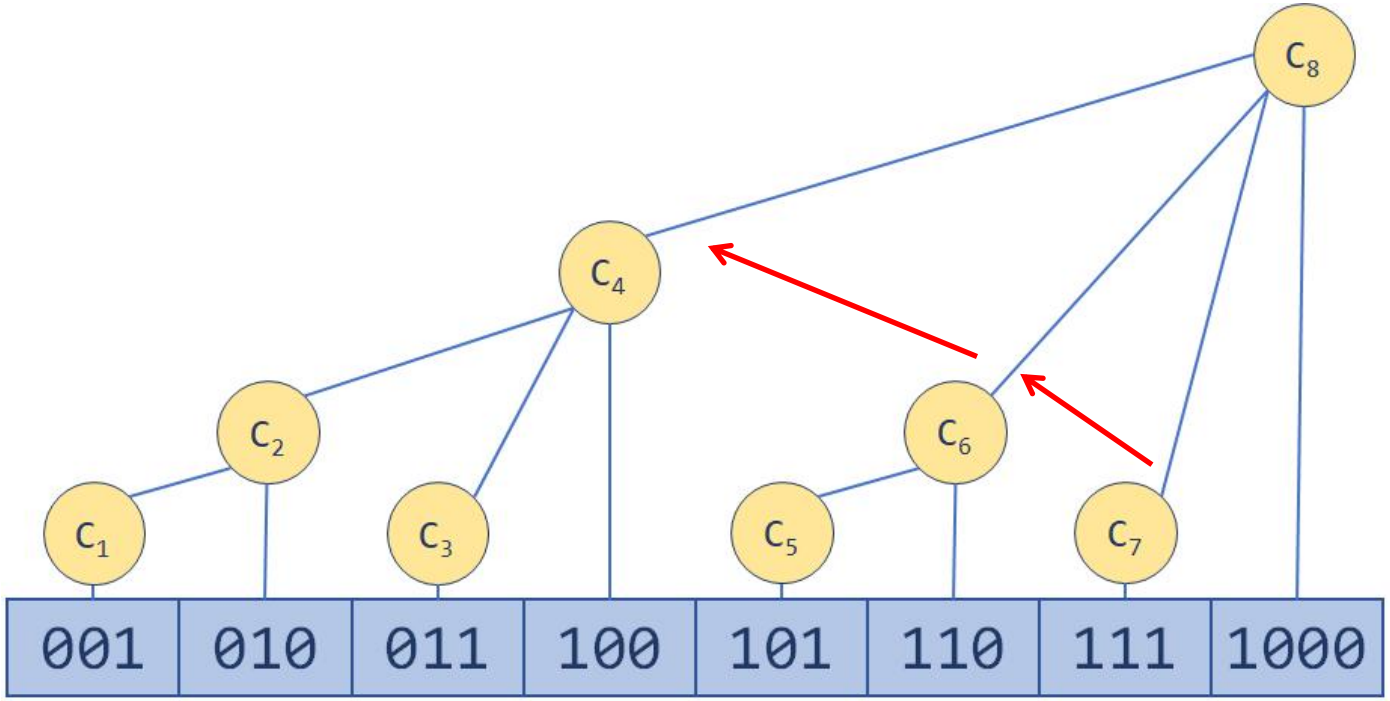
可以分别查询 (0,8]、(8,10]、(10,11] 区间数组元素的和再相加。这三个区间怎么来呢？其实就是不断的去掉二进制最右边的一个1的过程



树状数组的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

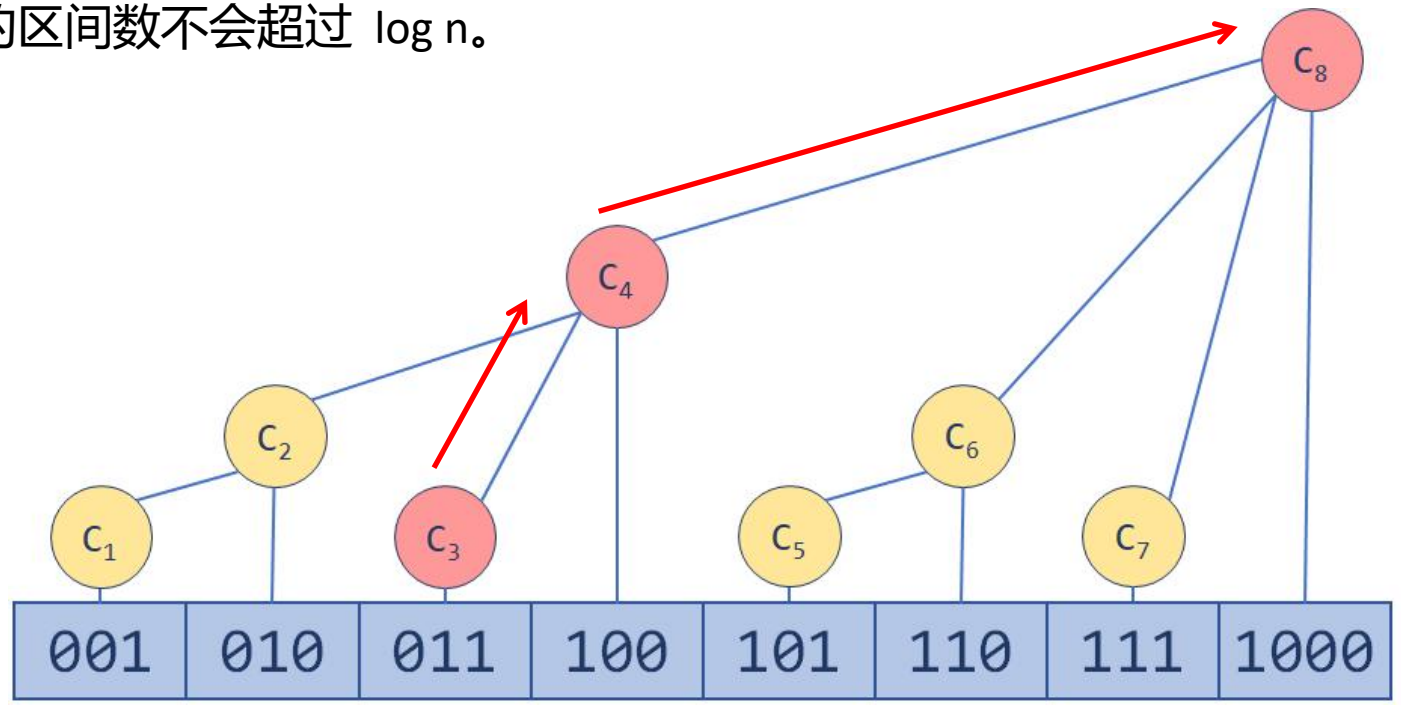
我们定义，二进制数x最右边的一个 1，连带着它之后的 0 为 lowbit(x)。那么我们用 C_i 维护区间 (i − lowbit(i)，i]，这样显然查询前n项和时需要合并的区间数是少于log(n)的。定义 C_i 的父节点为 $C_{i+lowbit(i)}$ ，所形成的树状数组的结构像下面这样：



树状数组的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

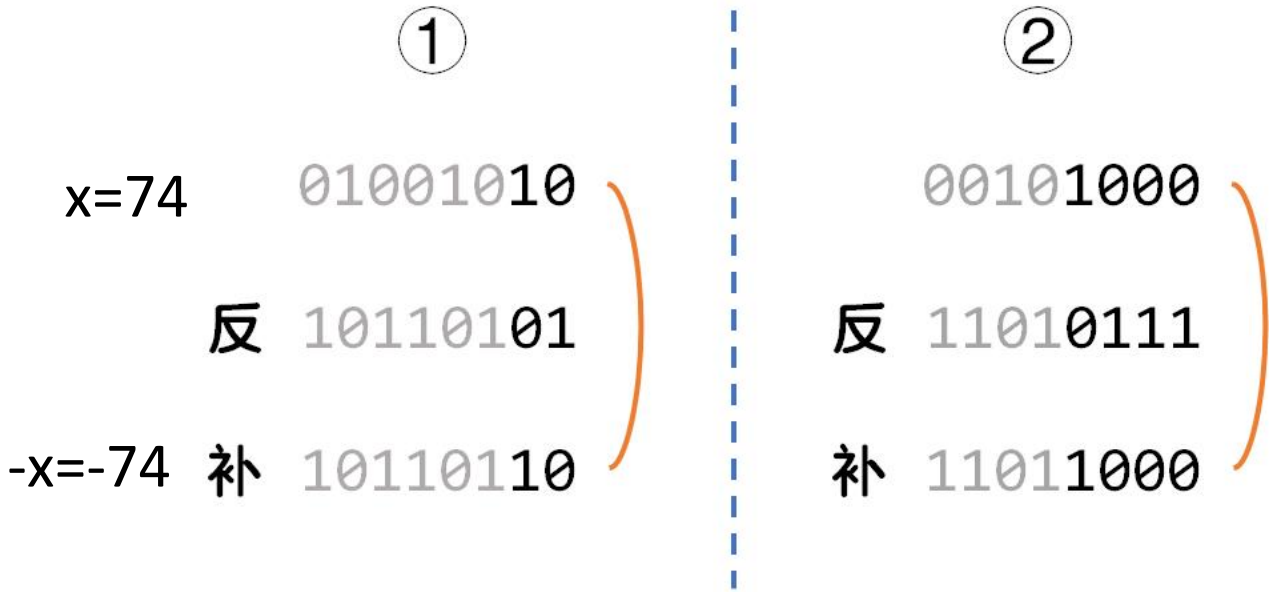
那么如何实现单点修改呢，大家会发现更新就是一个“**爬树**”的过程。一路往上更新，直到n（树状数组的容量）。每一步都把从右边起一系列连续的1变为0，再把这一系列1的前一位0变为1，每一次加的正是 lowbit(x)。这样，我们更新的区间数不会超过 log n。



树状数组的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

lowbit(x)定义为能整除 i 的最大的2的次幂，理解为将 i 转为二进制之后最后一个 1 的权重。lowbit(x)可以通过位运算x & -x 很快的计算。计算机里有符号数一般是以补码的形式存储的。-x相当于x按位取反再加1，会把结尾处原来1000...的形式，变成0111...，再变成1000...；而前面每一位都与原来相反。这时我们再把它和x按位与，得到的结果便是lowbit(x)。下面的图中举了两个例子：



COOLMAKER 酷哥创客 人工智能教育

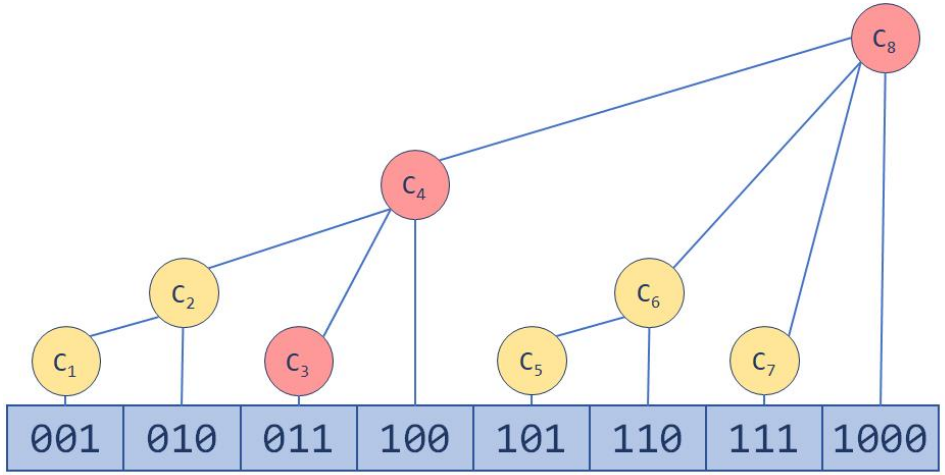
获取x的最右边的1

单点修改

求前x项的和

区间查询

```
6   struct BIT
7   {
8       ll c[maxn];//树状数组
9       int lowbit(int x)//获取最右边的1
10      {
11          return x&(-x);
12      }
13      void update(int x,int k)//单点修改
14      {
15          for(int i=x; i<=n; i+=lowbit(i))
16          {
17              c[i]+=k;
18          }
19      }
20      ll query(int x)//求1-x的和
21      {
22          ll ans=0;
23          for(int i=x; i>0; i-=lowbit(i))
24          {
25              ans+=c[i];
26          }
27          return ans;
28      }
29      ll query(int l,int r)//区间查询【L,R】的和
30      {
31          return query(r)-query(l-1);
32      }
33  } tree;
```



3274: 【模板】树状数组单点修改、区间查询

时间限制: 2 Sec 内存限制: 256 MB

提交: 0 解决: 1

[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★☆☆

题目描述

这是一道模板题。

给定数列 $a_1, a_2, \ldots, a_n$，你需要依次进行 q 个操作，操作有两类：

- `1 i x`：给定 i, x，将 a_i 加上 x；
- `2 l r`：给定 l, r，求 $\sum_{i=l}^{r} a_i$ 的值（换言之，求 $a_l + a_{l+1} + \cdots + a_r$ 的值）。

输入

第一行包含 2 个正整数 n, q，表示数列长度和询问个数。保证 $1 \leq n, q \leq 10^6$。
第二行 n 个整数 $a_1, a_2, \ldots, a_n$，表示初始数列。保证 $|a_i| \leq 10^6$。
接下来 q 行，每行一个操作，为以下两种之一：

- `1 i x`：给定 i, x，将 $a[i]$ 加上 x；
- `2 l r`：给定 l, r，求 $\sum_{i=l}^{r} a_i$ 的值。

保证 $1 \leq l \leq r \leq n, |x| \leq 10^6$。

输出

对于每个 `2 l r` 操作输出一行，每行有一个整数，表示所求的结果。

样例输入

```
3 2
1 2 3
1 2 0
2 1 3
```

样例输出

```
6
```

来源/分类

树状数组

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn=1e6+5;
int n,q;// n代表数组的范围 q代表询问次数
struct BIT
{
    ll c[maxn];//树状数组
    int lowbit(int x)//获取最右边的1
    {...}
    void update(int x,int k)//单点修改
    {...}
    ll query(int x)//求1-X的和
    {...}
    ll query(int l,int r)//区间查询【L,R】的和
    {...}
} tree;
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    cin>>n>>q;
    int op,i,x,l,r;
    for(int i=1; i<=n; i++)
    {
        cin>>x;
        tree.update(i,x);
    }
    while(q--)
    {
        cin>>op;
        if(op==1)
        {
            cin>>i>>x;
            tree.update(i,x);
        }
        else
        {
            cin>>l>>r;
            cout<<tree.query(l,r)<<"\n";
        }
    }
    return 0;
}
```

3275: 【模板】树状数组区间修改、单点查询

时间限制: 2 Sec 内存限制: 512 MB
提交: 0 解决: 2
[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★☆☆

题目描述

这是一道模板题。

给定数列 $a[1], a[2], \ldots, a[n]$，你需要依次进行 q 个操作，操作有两类：

- `1 l r x`：给定 l, r, x，对于所有 $i \in [l, r]$，将 $a[i]$ 加上 x（换言之，将 $a[l], a[l+1], \ldots, a[r]$ 分别加上 x）；
- `2 i`：给定 i，求 $a[i]$ 的值。

输入

第一行包含 2 个正整数 n, q，表示数列长度和询问个数。保证 $1 \leq n, q \leq 10^6$。
第二行 n 个整数 $a[1], a[2], \ldots, a[n]$，表示初始数列。保证 $|a[i]| \leq 10^6$。
接下来 q 行，每行一个操作，为以下两种之一：

- `1 l r x`：对于所有 $i \in [l, r]$，将 $a[i]$ 加上 x；
- `2 i`：给定 i，求 $a[i]$ 的值。

保证 $1 \leq l \leq r \leq n, |x| \leq 10^6$。

这看起来与单点修改&区间查询差不多，但实际上有很大的区别。
修改一个区间我需要进行R-L+1次，

输出

对于每个 `2 i` 操作，输出一行，每行有一个整数，表示所求的结果。

样例输入

```
3 2
1 2 3
1 1 3 0
2 2
```

样例输出

```
2
```

来源/分类
树状数组

[提交] [状态][题解] [Edit] [TestData]

树状数组的区间修改、单点查询

COOLMAKER
酷哥创客 人工智能教育

突破口：差分

举个例子：记差分数组为 T。

原来的序列为：

1	2	3	4	5

差分数组为：

1	1	1	1	1

在 $[2,4]$ 之间 $+2$，则序列为：

1	4	5	6	5

差分数组变为：

1	3	1	1	-1

对比发现，只有 T_2+2，以及 T_5-2。只用了两次单点修改。



```
struct BIT2
{
    ll c[maxn];//树状数组
    int lowbit(int x)
    {
        return x&(-x);
    }
    void update(int x,int k)
    {
        for(int i=x; i<=n; i+=lowbit(i))
        {
            c[i]+=k;
        }
    }
    void update(int l,int r,int k)
    {
        update(l,k);
        update(r+1,-k);
    }
    ll query(int x)//求1-n的和
    {
        ll ans=0;
        for(int i=x; i>0; i-=lowbit(i))ans+=c[i];
        return ans;
    }
} tree;
```

3275: 【模板】树状数组区间修改、单点查询

时间限制: 2 Sec 内存限制: 512 MB
提交: 0 解决: 2
[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★☆☆

题目描述

这是一道模板题。

给定数列 $a[1], a[2], \ldots, a[n]$，你需要依次进行 q 个操作，操作有两类：

- `1 l r x`：给定 l, r, x，对于所有 $i \in [l, r]$，将 $a[i]$ 加上 x（换言之，将 $a[l], a[l+1], \ldots, a[r]$ 分别加上 x）；
- `2 i`：给定 i，求 $a[i]$ 的值。

输入

第一行包含 2 个正整数 n, q，表示数列长度和询问个数。保证 $1 \le n, q \le 10^6$。
第二行 n 个整数 $a[1], a[2], \ldots, a[n]$，表示初始数列。保证 $|a[i]| \le 10^6$。
接下来 q 行，每行一个操作，为以下两种之一：

- `1 l r x`：对于所有 $i \in [l, r]$，将 $a[i]$ 加上 x；
- `2 i`：给定 i，求 $a[i]$ 的值。

保证 $1 \le l \le r \le n, |x| \le 10^6$。

输出

对于每个 `2 i` 操作，输出一行，每行有一个整数，表示所求的结果。

样例输入

```
3 2
1 2 3
1 1 3 0
2 2
```

样例输出

```
2
```

来源/分类
树状数组

[提交] [状态][题解] [Edit] [TestData]

插入的是原数组的差分数组

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn=1e6+5;
int n,q;
struct BIT2
{
    ll c[maxn];//树状数组
    int lowbit(int x)
    {...}
    void update(int x,int k)
    {...}
    void update(int l,int r,int k)
    {...}
    ll query(int x)//求1-X的和
    {...}
} tree;
int a[maxn];
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    cin>>n>>q;
    int op,i,x,l,r;
    for(int i=1; i<=n; i++)
    {
        cin>>a[i];
        tree.update(i,a[i]-a[i-1]);
    }
    while(q--)
    {
        cin>>op;
        if(op==1)
        {
            cin>>l>>r>>x;
            tree.update(l,r,x);
        }
        else
        {
            cin>>i;
            cout<<tree.query(i)<<"\n";
        }
    }
    return 0;
}
```

3276: 【模板】树状数组区间修改、区间查询

时间限制: 2 Sec 内存限制: 256 MB
提交: 0 解决: 2
[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestData]

难度

★★★★☆

题目描述

这是一道模板题。

给定数列 $a[1], a[2], \ldots, a[n]$，你需要依次进行 q 个操作，操作有两类：

- `1 l r x`：给定 l, r, x，对于所有 $i \in [l, r]$，将 $a[i]$ 加上 x（换言之，将 $a[l], a[l+1], \ldots, a[r]$ 分别加上 x）；
- `2 l r`：给定 l, r，求 $\sum_{i=l}^{r} a[i]$ 的值（换言之，求 $a[l] + a[l+1] + \cdots + a[r]$ 的值）。

输入

第一行包含 2 个正整数 n, q，表示数列长度和询问个数。保证 $1 \leq n, q \leq 10^6$。
第二行 n 个整数 $a[1], a[2], \ldots, a[n]$，表示初始数列。保证 $|a[i]| \leq 10^6$。
接下来 q 行，每行一个操作，为以下两种之一：

- `1 l r x`：对于所有 $i \in [l, r]$，将 $a[i]$ 加上 x；
- `2 l r`：输出 $\sum_{i=l}^{r} a[i]$ 的值。

保证 $1 \leq l \leq r \leq n, |x| \leq 10^6$。

输出

对于每个 `2 l r` 操作，输出一行，每行有一个整数，表示所求的结果。

样例输入

```
5 10
2 6 6 1 1
2 1 4
1 2 5 10
2 1 3
2 2 3
1 2 2 8
1 2 3 7
1 4 4 10
2 1 2
1 4 5 6
2 3 4
```

样例输出

```
15
34
32
33
50
```

来源/分类
树状数组 差分

这看起来与区间修改&单点查询差不多，但实际上有很大的区别。
查询一个区间的和我需要进行R-L+1次，

COOLMAKER 酷哥创客 人工智能教育

树状数组的区间修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

在 区间修改&单点查询 问题中我们的优化是使用树状数组维护**差分序列**
我们现在在此基础上进行如下推导：
设A数组为**原数组**
S数组为A数组的**前缀和**
C数组为A数组的**差分数组**

A1=C1
A2=C1+C2
A3=C1+C2+C3
.....
An=C1+C2+C3+C4+...+Cn

Sn =A1+A2+A3+...+An
=C1*n+C2*(n-1)+C3*(n-2)+...+Cn
=(n+1)*(C1+C2+C3+...+Cn) - (C1+2*C2+3*C3+...+n*Cn)

此时 C1+C2+C3+...+Cn 就是我们之前区间修改&单点查询 问题中的**差分序列前缀和**，也就是**An**的值，而现在在区间查询问题中，我们需要在之前的基础上**额外维护一个i*Ci的树状数组**，根据如上公式做差即可得到 [1 , n]区间的和



3276: 【模板】树状数组区间修改、区间查询

时间限制: 2 Sec 内存限制: 256 MB

难度

★★★★☆

题目描述

这是一道模板题。

给定数列 $a[1], a[2], \ldots, a[n]$，你需

- `1 l r x`：给定 l, r, x，对于所
- `2 l r`：给定 l, r，求 $\sum_{i=l}^{r} a[i]$

上 x）；

输入

第一行包含 2 个正整数 n, q，表示数
第二行 n 个整数 $a[1], a[2], \ldots, a[n]$
接下来 q 行，每行一个操作，为以下

- `1 l r x`：对于所有 $i \in [l, r]$
- `2 l r`：输出 $\sum_{i=l}^{r} a[i]$ 的值。

保证 $1 \le l \le r \le n, |x| \le 10^6$。

输出

对于每个 `2 l r` 操作，输出一行，

样例输入

```
5 10
2 6 6 1 1
2 1 4
1 2 5 10
2 1 3
2 2 3
1 2 2 8
1 2 3 7
1 4 4 10
2 1 2
1 4 5 6
2 3 4
```

样例输出

```
15
34
32
33
50
```

来源/分类

树状数组 差分

```
struct BIT3
{
    ll C[maxn],B[maxn];
    int lowbit(int x)
    {
        return x&(-x);
    }
    void update(int x,ll k)
    {
        for(int i=x; i<=n; i+=lowbit(i))
        {
            C[i]+=k;
            B[i]+=x*k;
        }
    }
    void update(int l,int r,ll k)
    {
        update(l,k);
        update(r+1,-k);
    }
    ll query(int x)
    {
        ll ans=0;
        for(int i=x; i>0; i-=lowbit(i))
        {
            ans+=(x+1)*C[i]-B[i];
        }
        return ans;
    }
    ll query(int l,int r)
    {
        return query(r)-query(l-1);
    }
} tree;
```

//维护差分数组的树状数组
//维护i*C[i]的树状数组

//Sn=(n+1)*(C1+C2+C3+...+Cn) - (C1+2*C2+3*C3+...+n*Cn)

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn=1e6+5;
int n,q;
struct BIT3
{
    ll C[maxn],B[maxn];
    int lowbit(int x)
    {...}
    void update(int x,ll k)
    {...}
    void update(int l,int r,ll k)
    {...}
    ll query(int x)
    {...}
    ll query(int l,int r)
    {...}
} tree;
int a[maxn];
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    cin>>n>>q;
    int op,x,l,r;
    for(int i=1; i<=n; i++)
    {
        cin>>a[i];
        tree.update(i,a[i]-a[i-1]);
    }
    while(q--)
    {
        cin>>op;
        if(op==1)
        {
            cin>>l>>r>>x;
            tree.update(l,r,x);
        }
        else
        {
            cin>>l>>r;
            cout<<tree.query(l,r)<<"\n";
        }
    }
    return 0;
}
```

二维树状数组的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

二维树状数组，也被称作树状数组套树状数组(树套树)，也就是每一个节点都是一个树状数组，用来维护二维数组上的单点修改和前缀信息问题。一维树状数组维护的是一个一维的数列，那么二维树状数组就是维护了一个一个二维的矩阵了。

- 我们在进行单点修改操作的时候，对于第一维的树状数组，我们要考虑修改这个位置的树状数组会影响到多少个位置的树状数组，进入到第二维时考虑的就是这个位置修改后会影响到多少个位置。可以发现，这样操作后其实就是修改了一个一个的小矩形了，总的单次修改的时间复杂度为 O(log n * log m)

```
void update(int x,int y,int k)
{
    for(int i=x; i<=n; i+=lowbit(i))
    {
        for(int j=y; j<=m; j+=lowbit(j))
        {
            c[i][j]+=k;
        }
    }
}
```

二维树状数组的单点修改、区间查询

COOLMAKER
酷哥创客 人工智能教育

- 单点修改操作的本质其实是跟一维树状数组是一样的，那么查询操作也是同理，依然是考虑有多少个小矩形要加。当然，这次查询的，就是二维前缀和了，时间复杂度为O(log n * log m)

```
ll query(int x,int y)
{
    ll ans=0;
    for(int i=x; i>0; i-=lowbit(i))
    {
        for(int j=y; j>0; j-=lowbit(j))
        {
            ans+=c[i][j];
        }
    }
    return ans;
}
```

- 区间查询也是像二维前缀和数组一样利用**容斥原理**算出这个小矩形的和

query(x2,x2)-query(x1-1,y2)-query(x2,y1-1)+query(x1-1,y1-1);

```
ll query(int x1,int y1,int x2,int y2)
{
    return query(x2,y2)-query(x1-1,y2)-query(x2,y1-1)+query(x1-1,y1-1);
}
```



3277: 【模板】二维树状数组单点修改、区

时间限制: 3 Sec 内存限制: 256 MB

提交: 0 解决: 2

[提交] [状态] [讨论版] [命题人: 18958865698] [Edit] [TestDa

难度

```
struct BIT
{
    ll c[maxn][maxn];//树状数组
    int lowbit(int x)
    {
        return x&(-x);
    }
    void update(int x,int y,int k)
    {
        for(int i=x; i<=n; i+=lowbit(i))
        {
            for(int j=y; j<=m; j+=lowbit(j))
            {
                c[i][j]+=k;
            }
        }
    }
    ll query(int x,int y)
    {
        ll ans=0;
        for(int i=x; i>0; i-=lowbit(i))
        {
            for(int j=y; j>0; j-=lowbit(j))
            {
                ans+=c[i][j];
            }
        }
        return ans;
    }
    ll query(int x1,int y1,int x2,int y2)
    {
        return query(x2,y2)-query(x1-1,y2)-query(x2,y1-1)+query(x1-1,y1-1);
    }
} tree;
```

(c,d) 的子矩阵内所

问答。

样例输出

```
7
```

来源/分类

二维树状数组

[提交] [状态][题解] [Edit] [TestData]

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn=4e3+100;
int n,m,q;
struct BIT
{
    ll c[maxn][maxn];//树状数组
    int lowbit(int x)
    {...}
    void update(int x,int y,int k)
    {...}
    ll query(int x,int y)
    {...}
    ll query(int x1,int y1,int x2,int y2)
    {...}
} tree;
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    cin>>n>>m;
    int op,x,y,k,a,b,c,d;
    while(cin>>op)
    {
        if(op==1)
        {
            cin>>x>>y>>k;
            tree.update(x,y,k);
        }
        else
        {
            cin>>a>>b>>c>>d;
            cout<<tree.query(a,b,c,d)<<"\n";
        }
    }
    return 0;
}
```



树状数组维护不可差分信息

- 树状数组除了**维护区间和**，**区间异或**等信息外，还可以用来维护一些不可差分的信息。例如维护区间最值，区间GCD等等
- 注意，这种方法虽然码量小，但单点修改和区间查询的时间复杂度均为 O(log n*log n)，比使用线段树的时间复杂度 O(log n) 劣。



树状数组维护不可差分信息

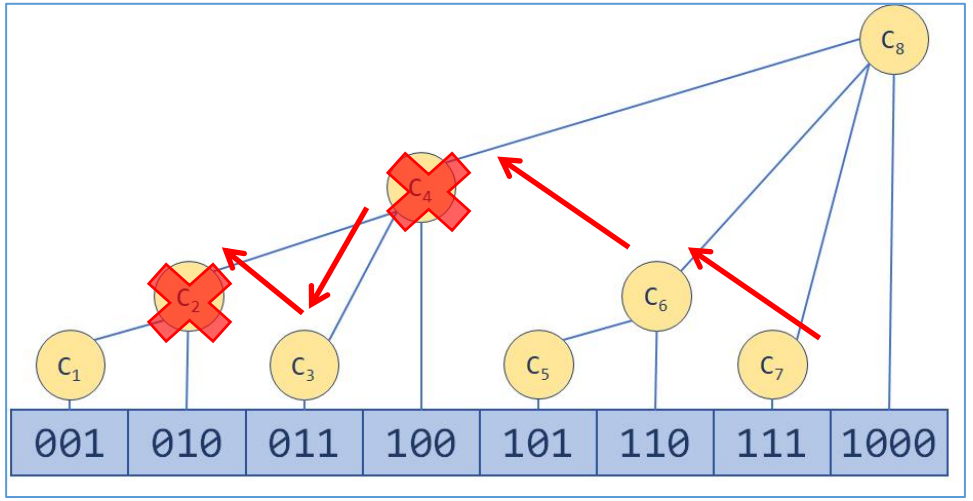
区间查询

我们还是基于之前的思路，从 R 沿着 lowbit 一直向前跳，但是我们不能跳到 L 的左边。
因此，如果我们跳到了 c[x]，先判断下一次要跳到的 x - lowbit(x) 是否小于 L：

- 如果小于 L，我们直接把 a[x]单点 合并到总信息里，然后跳到 c[x - 1]。
- 如果大于等于 R，说明没越界，正常合并 c[x]，然后跳到 c[x - lowbit(x)] 即可。

下面以查询区间最大值为例，给出代码：

```
ll getmax(int l, int r)
{
    ll ans = 0;
    while(r >= l)
    {
        ans = max(ans, a[r]);
        --r;
        for(; r - lowbit(r) >= l; r -= lowbit(r))
        {
            ans = max(ans, C[r]);
        }
    }
    return ans;
}
```



树状数组维护不可差分信息

COOLMAKER
酷哥创客 人工智能教育

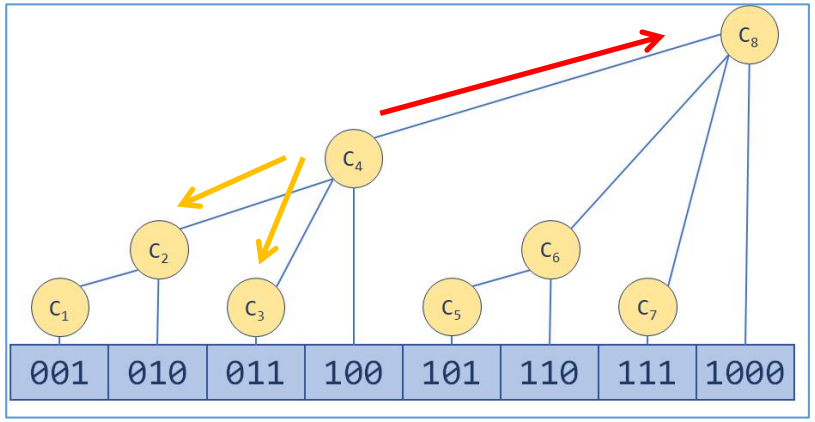
单点更新

对于可差分信息需要更新满足在树状数组树形态上，所有满足 y 是 x 的祖先的 c[y]。

而对于不可差分信息，不存在通过 p 直接修改 c[y] 的方式。这是因为修改本身就相当于是把旧数从原区间「移除」，然后加入一个新数。「移除」时对区间信息的影响，相当于做「逆运算」，而不可差分信息不存在「逆运算」，所以无法直接修改 c[y]。换句话说，对每个受影响的 c[y]，这个区间的信息我们必定要重构了。

考虑 c[y] 的儿子们，它们的信息一定是正确的（因为我们先更新儿子再更新父亲），而这些儿子又恰好组成了 [lowbit(y), y - 1] 这一段管辖区间，那再合并一个单点 a[y] 就可以合并出 [lowbit(y), y]，也就是 c[y] 了。这样，我们能用至多 log n 个区间重构合并出每个需要修改的 c。

```
void update(int x, ll v)
{
    a[x] = v;
    for(int i = x; i <= n; i += lowbit(i))
    {
        C[i] = a[i];
        for(int j = 1; j < lowbit(i); j *= 2)// 枚举受影响的区间
        {
            C[i] = max(C[i], C[i - j]);
        }
    }
}
```



3284: 区间最大值

时间限制: 2 Sec 内存限制: 256 MB

提交: 0 解决: 1

难度

★★★☆☆

题目描述

输入一串数字，给你 X 到 Y 这

输入

第一行两个整数 N,
接下来一行为 N 个
接下来 M 行，每行

输出

输出共 M 行，每行

样例输入

```
10 2
3 2 4 5 6 8 1 2 9 7
1 4
3 8
```

样例输出

```
5
8
```

来源/分类

树状数组维护不可差分信息

```cpp
struct BITmax
{
    ll a[maxn],C[maxn];
    int lowbit(int x)
    {
        return x&(-x);
    }
    void update(int x, ll v)
    {
        a[x] = v;
        for(int i = x; i <= n; i += lowbit(i))
        {
            // 枚举受影响的区间
            C[i] = a[i];
            for(int j = 1; j < lowbit(i); j *= 2)
            {
                C[i] = max(C[i], C[i - j]);
            }
        }
    }
    ll getmax(int l, int r)
    {
        ll ans = 0;
        while(r >= l)
        {
            ans = max(ans, a[r]);
            --r;
            for(; r - lowbit(r) >= l; r -= lowbit(r))
            {
                // 注意，循环条件不要写成 r - lowbit(r) + 1 >= l
                // 否则 l = 1 时，r 跳到 0 会死循环
                ans = max(ans, C[r]);
            }
        }
        return ans;
    }
} tree;
```

```cpp
#include <bits/stdc++.h>
using namespace std;
#define IO ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);
typedef long long ll;
const int maxn=1e5+5;
int n,q;
struct BITmax
{
    ll a[maxn],C[maxn];//a原数组，C树状数组维护子树内节点最大值
    int lowbit(int x)
    {...}
    void update(int x, ll v)
    {...}
    ll getmax(int l, int r)
    {...}
} tree;
int main()
{
    IO;
    cin>>n>>q;
    for(int i=1,x; i<=n; i++)
    {
        cin>>x;
        tree.update(i,x);
    }
    int l,r;
    while(q--)
    {
        cin>>l>>r;
        cout<<tree.getmax(l,r)<<"\n";
    }
    return 0;
}
```